# DESIGN AND DEVELOPMENT OF A MULTI-GAME LEARNING PLATFORM FOR COMPUTATIONAL THINKING

**MD SHAKIB MOSTAFA**

**BACHELOR IN COMPUTER SCIENCE (HONOURS)**

**ALBUKHARY INTERNATIONAL UNIVERSITY**

**SEPTEMBER 2026**

# DESIGN AND DEVELOPMENT OF A MULTI-GAME LEARNING PLARTFORM FORCOMPUTATIONAL THINKING

MD SHAKIB MOSTAFA

Report Submitted to Fulfill the Partial Requirements for Bachelor in Computer Science (Honours)

ALBUKHARY INTERNATIONAL UNIVERSITY
ALOR SETAR, MALAYSIA

September 2026

# DECLARATION

I declare that this dissertation is of my original work and all references have been cited adequately as required.

Name: MD SHAKIB MOSTAFA

# APPROVAL

This thesis has been approved in partial fulfillment for the Bachelor of Computer Science (Honours)

| | |
|---|---|
| Thesis Supervisor | Name: …… |
| | Official Stamp: ……. |
| Final Year Project Coordinator | Name: …… |
| | Official Stamp: ……. |
| Department Dean | Name: …… |
| | Official Stamp: ……. |

## ACKNOWLEDGMENT

I would like to express my sincere gratitude to my supervisors, Dr Qudrattullah Omerkhel, for their guidance, support and valuable feedback throughout this project. Their comments helped me improve the project idea, scope, methodology and system diagrams.

I would also like to thank the School of Computing and Informatics at Albukhary International University for giving me the opportunity to undertake this project. Through this project, I gained a better understanding of how to plan and organize a computer science project in a structured and effective way.

# TABLE OF CONTENTS


**DECLARATION** **I**
**APPROVAL** **II**
**ACKNOWLEDGMENT** **III**
**TABLE OF CONTENTS** **IV**
**LIST OF FIGURES** **VII**
**LIST OF TABLES** **VIII**
**ABBREVIATIONS** **IX**
**ABSTRACT** **X**
**Chapter One** **1**
**Introduction** **1**
1.1 Project Background 1
1.2 Problem Statement 1
1.3 Project Objectives 2
1.4 Scope of the Project 2
1.5 Significance of the Project 4
1.6 Report Organization 5
**Chapter Two** **7**
**Literature Review** **7**
2.1 Introduction 7
2.2 Computational Thinking 7
2.3 Game-Based Learning and Active Problem-Solving 9
2.4 Interactive Algorithm Visualization and Scaffolding 9
2.5 Gamification, Feedback and Learner Motivation 10
2.6 Review and Comparison of Existing Educational Platforms 11
2.6.1 Limitations of Existing Approaches 11
2.6.2 Comparison of Existing Platforms 12
2.6.3 Research Gap and Project Response 13
2.6.4 Project Originality 14
2.7 Summary 14
**Chapter Three** **15**
**Methodology and System Design** **15**
3.1 Introduction 15
3.2 Development Methodology 15
3.3 Development Sprints 16
3.4 System Architecture 17
3.5 Behavioral Diagrams 20
3.5.1 Use-Case Diagram 21
3.5.2 Activity Diagram 22
3.5.3 Class Diagram 25
3.5.4 Sequence Diagram 25
3.6 System Requirements 28

3.6.1 Functional Requirements 28
3.6.2 Non-Functional Requirements 29
3.7 Hardware Requirements 30
3.8 Software Requirements 31
3.9 Summary 32
**Chapter Four 33**
**Implementation and Evaluation 33**
4.1 Implementation Overview 33
4.2 Algorithms and Data Structures 34
4.2.1 Bubble Sort 34
4.2.2 Tic-Tac-Toe 38
4.2.3 Dijkstra's Pathfinding 40
4.2.4 Three Men's Morris 43
4.2.5 Chess 47
4.3 Shared Application Services 50
4.3.1 Platform State and Persistence 50
4.3.2 SoundSynth and Preferences 51
4.3.3 Three.js Background Infrastructure 52
4.4 Responsive Interface and Guidance 52
4.5 Testing and Evaluation 53
4.5.1 Testing Evidence and Scope 53
4.5.2 Functional and Non-Functional Testing 53
4.5.3 Functional Test Cases 54
4.5.4 Proposed User-Evaluation Survey 59
4.6 Summary 60
**Chapter Five 61**
**Conclusion and Discussion 61**
5.1 Overview 61
5.2 Project Practical Contribution 61
5.3 Contribution to Social Business 65
5.4 Roles and Responsibilities 66
5.5 Discussion 66
5.6 Project Limitations 70
5.7 Future Work 71
5.7.1 Improving the Current System 71
5.7.2 Improving Progress Tracking 71
5.7.3 Adding More Learning Activities 72
5.7.4 Adaptive Difficulty and Feedback 73
5.7.5 Testing, Accessibility and Deployment 73
5.7.6 Future Educational Evaluation 74
5.8 Recommendations 74
5.9 Conclusion 75
**References 77**

**Appendix A: Project Schedule** **84**
**Appendix B: Running the Platform** **85**
**Appendix C: System Screenshots** **89**

# LIST OF FIGURES

Figure 3.1: System Architecture Diagram 19
Figure 3.2: Use Case Diagram 21
Figure 3.3: Activity Diagram 23
Figure 3.4: Class Diagram 24
Figure 3.5: Sequence Diagram 26
Figure A.1: Planned development and documentation schedule 82
Figure C.1: Multi-Game Learning Platform Dashboard 87
Figure C.2: Bubble Sort Activity 87
Figure C.3: Dijkstra's Pathfinding Activity 88
Figure C.4: Tic-Tac-Toe Minimax Evaluation Panel 88
Figure C.5: Three Men's Morris Activity 89
Figure C.6: Chess Activity 89
Figure C.7: Light and Dark Theme Interfaces 90

# LIST OF TABLES

Table 1.1: Games and computational-thinking concepts 3
Table 2.1: Comparison of existing platforms 12
Table 3.1: Scrum sprint activities and outcomes 17
Table 3.2: Game modes and user controls 27
Table 3.3: Functional requirements 28
Table 3.4: Non-functional requirements 29
Table 3.5: Hardware requirements 30
Table 3.6: Software requirements 31
Table 4.1: Source-file responsibilities 33
Table 4.2: Shared progress updates by activity 51
Table 4.3: Functional test cases for the Multi-Game Learning Platform 55
Table 4.4: Proposed user-evaluation questionnaire 60
Table 5.1: Practical contributions and current implementation boundaries 65
Table 5.2: Roles and responsibilities 68
Table B.1: File placement for local execution 84

# ABBREVIATIONS

AI: artificial intelligence
CT: computational thinking
CDN: content delivery network
DOM: Document Object Model
HTTP: Hypertext Transfer Protocol
JSON: JavaScript Object Notation
LMS: learning management system
PvP: player versus player
SDG: Sustainable Development Goal
SUS: System Usability Scale
SVG: Scalable Vector Graphics
UI: user interface

## ABSTRACT

Computational thinking involves noticing patterns and figuring out steps to take to solve a problem. These concepts are hard to get your head around when explained in words and/or code. A Multi-Game Learning Platform has been created using the browser for this project, which is comprised of five different games: Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess. The platform enables students to practice making smart decisions, determine if the choice is a valid one, and work out the path and sort out the choices.

It is based on HTML, CSS and JavaScript and serves the files with a local http based on node.js. A dashboard showing the five games. It also features instructions, feedback noises from the browser, light and dark mode as well as saved progress via localStorage. The data stored is: shared score, level, game data selection, theme and automatic help.

This platform was subjected to 24 documented functional test cases covering start-up, platform navigation, five learning activities, shared controls and browser storage. All 24 tests passed, giving a functional-test pass rate of 100%. No complete non-functional evaluation or user survey was conducted, so the report does not claim universal reliability, accessibility compliance, confirmed usability or improvement in computational-thinking ability.

During the implementation review and functional testing, the principal features of the platform, such as Chess AI, undo behavior, instructions and a Three.js background, were verified. The previously identified Tic-Tac-Toe candidate-score display problem was corrected by adding an evaluation panel and connecting it to the existing Minimax output. FT14 was repeated and passed. The project therefore produced a working educational prototype, while non-functional evaluation and future learning studies remain necessary.


***Keywords: algorithm visualization, game based learning, browser based learning, localStorage, JavaScript, computational thinking***

# CHAPTER ONE

# INTRODUCTION

## 1.1 Project Background

Problem solving is one of the most important aspects of computer science curriculum, and computational thinking has become an important element of that problem-solving process (Wu et al., 2024). It assists students to divide problem into smaller steps, see patterns, and provides clear and step-by-step instructions (Hamidi et al., 2023). Teaching computational thinking isn't teaching a particular syntax of code, it's teaching broader habits of analytical thinking such as abstraction, logical reasoning and algorithmic thinking.

It is difficult for beginners to grasp the ideas, particularly if they are being explained by the use of lecture or abstract examples from books (Liu et al., 2024). Errors and the strict compiler instructions cause a cognitive load and lead to focus away from the algorithmic logic itself (Lee et al., 2024). In the case of learning, it is hard to understand the workings of the sorting algorithms, the workings in the graph in graph-search algorithms, or the workings of state-space search without actually trying it yourself, but there is no way to interact with this with a standard code editor (Hassan & Zilles, 2024).

Game-based learning is a combination of theory and practice that consist of problem structures which will be challenging and an engaging activity which is low pressure (Videnovik et al., 2023). Feedback and direct manipulation can help in the development of mental model of the functioning of the algorithm. This active learning through an algorithm yields much more understanding than passive learning (Sibia et al., 2025). Designing multiple related tasks in a very simple interface also helps keep students motivated and supports the understanding of computational thinking in a variety of problem domains (López-Fernández et al., 2023).

To support the practical development of computational-thinking skills, this project introduces a web-based platform that combines algorithm visualization with strategy-based learning games.

## 1.2 Problem Statement

Students taking computer science (CS) for the first time, typically acquire the concepts of computational thinking by textually reading about the concept in the textbook or by engaging in coding drills and exercises focused on syntactic learning (Tikva & Tambouris, 2021). They know the name of an algorithm and the general purpose but they have difficulties to describe the changes to the state of a system made by the steps of the algorithm (Hayatpur et al., 2023). Basic operations such as

sorting an array, finding the shortest path or evaluating a move in a game are inherently visual and state-driven, meaning that they are inherently hard to understand without doing them manually (Sibia et al., 2025). Passive animations also don't help much as they do not push a learner to make decisions.

To cap it off there is a lot of fragmentation in tools used. A majority of them only have an interest in one sort of topic sorting or pathfinding and students waste mental energy in learning another interface instead of learning the concept itself (Sweller, 2024). So far there is not a single learning environment which can be used by entry-level students to study sorting and graph traversal at the same time together with board-game strategy (Zhang et al., 2024).

The purpose of this project is to fill this gap, and thus to make a single dashboard multi-game. It integrates manual step-throughs of algorithms with strategic board games to provide students with a variety of computational-thinking experiences all within a single, integrated, browser-based application (Videnovik et al., 2023).

### 1.3 Project Objectives

1. To design and develop a multi-game learning platform that integrates algorithm-based and strategy-based activities related to computational thinking.
2. To develop interactive activities that allow users to practice sorting, shortest-path, logical and strategic reasoning.
3. To implement a browser-based progress system that records supported scores, unlocked levels and selected game statistics.

### 1.4 Scope of the Project

This project focuses on the development of a client-side web application using HTML5, CSS3 and JavaScript (ES2022). The platform is mainly designed for desktop web browsers, while CSS Flexbox and Grid provide basic responsiveness on smaller screens. A lightweight Node.js HTTP server (server.js) serves the application files locally on port 3000. The system does not depend on an external backend framework, cloud infrastructure or external database.

This platform contains five learning games that connect game mechanics with selected computational-thinking skills. Bubble Sort and Dijkstra’s Pathfinding focus on algorithmic decision-making and visualization. Tic-Tac-Toe, Three Men’s Morris and Chess support logical reasoning, pattern recognition, state-space search and strategic planning. Table 1.1 summarizes the target skills and implemented features of each activity.

**Table 1.1: Games and Computational-Thinking Mapping**

| Game module | Target computational-thinking skill | Implemented features |
|---|---|---|
| **Bubble Sort** | Comparison and algorithmic steps | Arrays containing 3 to 20 elements, manual swap and advance controls, visual pass and swap counters, and immediate error feedback. |
| **Dijkstra's Pathfinding** | Weighted graphs and shortest paths | Interactive SVG and DOM graph, manual node selection, edge relaxation, distance table and three unlockable levels. |
| **Tic-Tac-Toe** | Game-state search and decision-making | A 3 × 3 board, Random AI, recursive Minimax AI and local player-versus-player mode. Candidate-move scores are displayed in the Minimax evaluation panel. |
| **Three Men's Morris** | Constraints and pattern recognition | A nine-position board with pre-placed pieces, adjacent-movement checks, basic computer moves and local player-versus-player mode. |
| **Chess** | Legal rules and strategic reasoning | An 8 × 8 board, legal-move validation, special moves, draw checks, one-ply and two-ply heuristic AI, local player-versus-player mode and undo history. |

These five games are integrated through a central dashboard. Shared platform features include instructional pop-ups, scoring, visual feedback, sound effects and light and dark themes. The Web Audio API produces supported sound feedback, while browser localStorage stores selected progress values, unlocked levels, theme preferences and help settings. Temporary activity states, such as unfinished board positions and move histories, are not restored after the page is reloaded.

This platform is intended to provide beginners with an accessible environment for practicing computational-thinking skills through algorithm visualization and strategy-based activities. However, completing an activity does not by itself prove an improvement in computational-thinking ability. A separate empirical study would be required to measure learning outcomes.

**Out-of-Scope Elements:**

These following elements were excluded because the project focused on developing and testing the core learning activities within the available time, resources and single-developer environment:

- **Remote user accounts, backend databases and server-side analytics:** These features require secure authentication, database administration, server hosting and additional privacy controls. The current platform uses browser localStorage instead.
- **Real-time online multiplayer and WebSocket communication:** Supporting live multiplayer would require a dedicated backend server, network-session management and additional security testing.

- **Native Android and iOS applications:** This project was developed as a browser-based application. Building separate mobile applications would require additional development tools, platform-specific design and testing.
- **Learning management system integration:** Connecting this platform to systems such as Moodle or Canvas would require external APIs, institutional access and additional integration testing.
- **Adaptive difficulty and machine-learning-based learner modelling:** These features would require sufficient learner-performance data and a validated model for adjusting activity difficulty.
- **Cross-device progress synchronization:** Progress is stored locally in the user's browser. Synchronizing it across devices would require user accounts, a cloud database and secure data transfer.
- **Formal usability and learning-outcome studies:** This proposed user-evaluation survey was not administered during the project. A learning-outcome study would also require participants, ethical approval where applicable, validated assessment instruments and sufficient time for data collection and analysis.

### 1.5 Significance of the Project

This project is significant because it combines several aspects of computational thinking within one interactive learning platform. Computational thinking includes skills such as problem decomposition, pattern recognition, abstraction, algorithmic reasoning and logical decision-making (Videnovik et al., 2023). Beginners may find these skills difficult to understand when they are presented only through written explanations or programming syntax.

The Bubble Sort and Dijkstra's Pathfinding activities allow learners to participate in selected steps of an algorithm. In Bubble Sort, learners decide whether adjacent values should be swapped or whether the comparison should continue. In Dijkstra's Pathfinding, they select nodes and calculate distance-relaxation values. This active involvement is intended to make algorithmic processes easier to follow than passive demonstrations alone (Mukasheva et al., 2023; Sibia et al., 2025).

Tic-Tac-Toe, Three Men's Morris and Chess provide opportunities to practice state-based and rule-based reasoning. These activities require learners to recognize patterns, identify legal moves, consider possible future states and make strategic decisions. The platform therefore represents computational thinking through both algorithm visualization and strategy-based activities.

All five activities are available through a shared dashboard. The platform also provides instructions, visual feedback, sound effects, light and dark themes, and selected scoring and progress information. These features are intended to make the activities clearer and more engaging.

This project also provides a modular foundation for future development. Its structure can support additional learning activities, broader browser and accessibility testing, and further refinement of the existing features.

Overall, this project demonstrates how different computational-thinking activities can be combined within one browser-based platform. Its main contribution is a working prototype that gives learners opportunities to practice algorithmic and logical reasoning. However, the project does not claim that the platform improves computational-thinking ability because this was not measured through a controlled educational study.

### 1.6 Report Organization

This report consists of five chapters, references and three appendices.

Chapter One provides an introduction to the project. It sets up the computational thinking and interactive learning world and describes the problem that the project aims to solve as well as the project objectives, scope and significance. It also describes main activities and constraints of the present prototype.

This literature review of Chapter Two covers the topics of computational thinking, gamification, game-based learning, algorithm visualization, scaffolding and learner motivation. It also examines some of the available platforms and provides a state of play on what was missing that this project addresses.

Chapter Three describes the methodology that was used in the development of the system and the system design. It shows how the platform has been developed step-by-step, the phases of the development, the client-side system architecture, the structural diagrams, behavioral diagrams, activity modes, functional requirements, non-functional requirements, and the hardware and software used to execute the platform.

This implementation and evaluation of the Multi-Game Learning Platform are discussed in Chapter Four. It includes a description of the source-file structure, the algorithms and data structures of the five activities, shared application services, browser storage, sound functions, theme controls, interface guidance and the Three.js background. It also presents the 24 functional test cases, the

corrected Tic-Tac-Toe candidate-score display, the boundaries of the non-functional evaluation and a proposed user-evaluation survey.

Chapter Five discusses the completed prototype, along with its practical contribution. It considers the social and educational relevance, roles and responsibilities, results of implementation, constraints and future work/recommendations of the project. At the end of the chapter an evaluation is made on the basis of the functions implemented, and evidence available at the time of submission.

References: Lists academic and technical references throughout the report. The project schedule is provided in Appendix A, and the folder structure and local use of the platform is provided in Appendix B. The Functional test cases and the results are provided in section 4.5.3 of Chapter Four.

# CHAPTER TWO

# LITERATURE REVIEW

## 2.1 Introduction

This chapter reviews literature on computational thinking, game-based learning, algorithm visualization, scaffolding, gamification and learner motivation. The aim of the Multi-Game Learning Platform (Wu et al., 2024) relies on the theoretical foundations of these inter-related areas. The review further looks at software platforms available to date to understand the current state of educational software and to illustrate the practical aspect this project aims to tackle (Zhang et al., 2024).

This platform developed in this project contains five activities: Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess. Bubble Sort and Dijkstra's Pathfinding require learners to make selected algorithmic decisions rather than only watching passive animations. This approach reflects research that highlights the value of active learner engagement in algorithm visualization (Hassan & Zilles, 2024; Sibia et al., 2025). The three board games provide opportunities to examine legal moves, compare possible actions and consider future board states through logical and strategic reasoning (Videnovik et al., 2023).

A design rationale for these specific activities and for how these activities can be combined in an integrated interface is provided in the literature (Sibia et al., 2025). But theory and literature do not necessarily argue that this type of platform would necessarily have a positive impact on the development of computational-thinking ability (Zhang et al., 2024). To assess the effectiveness of this as a learning method, a formal empirical study with validated learning assessment instruments would be needed.

## 2.2 Computational Thinking

Computational thinking is thought to be one of the important ways of thinking in computer science education. It assists students to decompose problems into parts and create solutions that can be implemented by humans or computers (Espinal et al., 2024; Palop et al., 2025). Computational thinking is not about syntax of programming languages, it is an analytical mode of thinking that can be applied in a range of fields (Montuori et al., 2024).

This study of the role of computational thinking in school curriculum has been the subject of many recent studies, including (Yeni et al. 2024), who identify key aspects of computational thinking as: abstraction, decomposition, algorithmic design and logical reasoning (Chen et al., 2023). Recent research describes computational thinking as involving connected concepts and practices that can

develop together through structured learning activities (Kong et al., 2025). It also extends beyond coding by encouraging learners to reason about systems, data and real-world problems (Grizioti, 2025). Their framework also suggests that computational thinking is not only about what is known, it is also the activities that materialize, test, and improve the reasoning of learners (Sibia et al., 2025).

In introductory computing education, computational thinking is often presented through core problem-solving processes such as decomposition, pattern recognition, abstraction and algorithmic thinking (Arslantaş, 2024). Decomposition means to divide a problem into smaller parts to be solved separately. The pattern recognition technique is to identify similarities between problems and abstraction to remove unnecessary details, so that learners can focus on the important aspects. Algorithmic thinking is the skill of being able to generate a step-by-step process (algorithm) that is clear enough to solve a problem (Arslantaş, 2024; Palop et al., 2025).

Text-based programming can place additional cognitive demands on beginners because they must learn programming keywords and syntax while also developing problem-solving skills (Mason & Seton, 2026).

To overcome this syntax hurdle, the games in this project, align to specific computational thinking concepts as follows:

- Bubble Sort Visualizer is a program that breaks up the sort of an array into the individual comparisons of elements. The learner may be seen as the logic machine that decides whether the neighboring values should be swapped or the pointer should be shifted towards the front.
- Path calculation was performed by Dijkstra's path finding models which are based on a weighted graph. It is abstracted based on the minimum-distance nodes that the learners are going to visit and distance relaxing of neighboring edges.
- Tic-Tac-Toe provides an example of state-space search and decision-making. It supports random AI moves and recursive Minimax evaluation, which calculates scores for candidate moves. The current interface displays the calculated root-move scores in an evaluation panel.
- Three Men’s Morris involves spatial pattern recognition and movement constraints . In this implementation, players move pre-placed pieces between adjacent positions on a board with nine positions, aiming to form a valid line of three pieces.
- Chess has a sufficiently large state space to allow for strategy to be planned. Learners play move filter, move rules, draw checks, heuristic board evaluation on a legal basis.

The platform maps each activity to selected analytical skills that learners are intended to practice. However, these mappings describe intended learning opportunities rather than confirmed educational

outcomes. Completing an activity alone does not demonstrate that a learner has mastered the associated computational-thinking concepts. Measuring improvement would require a formal empirical study using appropriate and validated computational-thinking assessment instruments (Zhang et al., 2024). Such an evaluation is outside the scope of the current project.

### 2.3 Game-Based Learning and Active Problem-Solving

Game-based learning uses games as structured learning environments to support engagement, knowledge development and skill practice (Videnovik et al., 2023). Its educational value depends on careful design so that gameplay, feedback and learning activities remain aligned with the intended learning objectives (Mikrouli et al., 2024). Research indicates that game-based learning can influence both cognitive outcomes, such as knowledge and skills, and affective outcomes, such as motivation and engagement (Hui & Mahmud, 2023). Challenges should also be balanced with learners' abilities to maintain engagement without creating unnecessary cognitive load (Matthew et al., 2025)

The platform applies game-based learning through active problem-solving. In the Bubble Sort and Dijkstra's Pathfinding activities, learners do not simply watch an algorithm run automatically. Instead, they make selected decisions that control its progress. Recent research suggests that interactive visualization can support learners' understanding of algorithm execution by allowing them to explore processes and observe changes in system state (Hassan & Zilles, 2024). Within the platform, an incorrect decision produces immediate visual feedback while the current step remains unchanged. Because learners interact through visual controls rather than writing code, they can practice the underlying reasoning without managing programming syntax or compiler messages.

Instructional pop-ups, highlighted elements and move indicators provide guidance within the platform. In the Bubble Sort activity, the two elements being compared are highlighted, and the learner selects either Swap Elements or Advance Pointer. Immediate feedback indicates whether the selected action is correct. In Dijkstra's Pathfinding, the interface displays tentative distances and provides feedback for node-selection and distance-relaxation decisions. These forms of scaffolding are intended to reduce unnecessary cognitive demands and help learners connect their actions with the underlying algorithmic logic (Sweller, 2024).

### 2.4 Interactive Algorithm Visualization and Scaffolding

Algorithm visualization presents the internal state and execution steps of an algorithm through graphical representations. However, displaying visual information alone does not guarantee conceptual understanding. Its educational value depends on the quality of the design and the level of learner engagement (Sibia et al., 2025). Visualization tools can provide more active learning

opportunities when learners predict outcomes, enter values, control execution steps and observe the resulting changes (Hassan & Zilles, 2024).

Active visualization is used in this platform in the Bubble Sort and Dijkstra's Pathfinding activities :

- **Bubble Sort:** The activity represents array values as numeric bars. The interface highlights the two elements currently being compared, displays the number of passes and swaps, and requires the learner to select either “Swap Elements” or “Advance Pointer.” This interactive approach allows learners to control individual steps of the sorting process rather than only watching an animation (Sibia et al., 2025).
- **Dijkstra’s Pathfinding:** The activity presents a weighted graph through an interactive visual interface. Learners select the unvisited node with the smallest tentative distance and enter the required distance-relaxation values. The graph and distance table are updated immediately, allowing learners to observe how their decisions contribute to the shortest path. This active interaction reflects recent research on the educational value of interactive algorithm visualization.
- **Strategy Games:** In Tic-Tac-Toe, the Minimax algorithm calculates candidate-move scores and displays them in an evaluation panel. This allows the learner to examine how the available root moves are assessed. In Chess and Three Men’s Morris, the interface highlights available moves after a piece is selected. These visual indicators help learners identify the legal actions available in the current board state.

Scaffolding is a structured aid that is given to the novice learner to understand new concepts and solve problems that they cannot easily do without the support (Sweller, 2024). For support, scaffolding is included in the platform in the form of instructional popups, the ability to visualize mathematical formulas, highlighting of moves, and predefined difficulty levels. The Dijkstra activity has three successively unlockable levels, Bubble Sort can have array sizes between 3 and 20 elements and Tic-Tac-Toe and Chess have selectable AI modes. Although these options can be used to scaffold the learner's practice, they are not adaptive, that is, it doesn't model the learner dynamically and adapt the tasks according to his/her performance.

### 2.5 Gamification, Feedback and Learner Motivation

Gamification involves using game-design elements, such as points, levels, challenges, rewards and feedback, within non-game learning activities. Recent meta-analyses indicate that gamification can improve learner motivation and engagement. However, its effects vary according to the design and learning context, and improvements in motivation do not always produce equal improvements in

competence or learning performance (Li et al., 2023; Yu et al., 2024). Gamification should therefore support clear learning objectives rather than being added only for entertainment.

This platform uses gamification in a targeted manner in its five modules:

- Recent research suggests that gamification elements such as points, levels and progress indicators can support learner motivation and engagement when they are meaningfully connected to learning activities (Li et al., 2023; Yu et al., 2024). In this project, Bubble Sort awards points according to the selected array size. Dijkstra's Pathfinding uses level progression and adjusts its completion reward according to mistakes and hint usage. Tic-Tac-Toe also awards points for selected outcomes. These scoring rules were designed specifically for the platform.
- There are three unlockable levels in Dijkstra's Pathfinding. Level is to unlock level 2, and learners know how much they have achieved in the activity.
- Audio feedback and persistence: In this case, the Web Audio API is utilized to create sounds for successful moves, incorrect moves as well as completion of the game. Using the browser localStorage, the scores, unlocked levels, theme preferences and help settings are all saved between browser sessions.

This completed functional tests confirmed that all five activities update their supported scores, levels and outcome statistics in browser storage. Tic-Tac-Toe records draws, while Three Men's Morris and Chess update their defined shared statistics. Temporary game states, including active board layouts, Chess undo histories and Dijkstra timer values, are intentionally cleared when the page is reloaded, so each activity begins a new session.

## 2.6 Review and Comparison of Existing Educational Platforms

### 2.6.1 Limitations of Existing Approaches

This research has identified several limitations in the use of algorithm-visualization software in computing education. Some tools rely mainly on passive animations, giving learners limited opportunities to interact with the algorithm or make decisions during its execution. As a result, learners may watch the visualization without fully understanding the underlying concepts (Hassan & Zilles, 2024; Sibia et al., 2025). Code-oriented tools may also require beginners to understand programming syntax before they can explore the algorithm, which can increase their cognitive load (Mason & Seton, 2026). In addition, some platforms focus on a single algorithm or type of activity. Learners may therefore need to use different tools and interfaces to practice a wider range of computational-thinking skills.

### 2.6.2 Comparison of Existing Platforms

Some of the existing ones provide a good example of the current approach to learning in Web based computing:

- VisuAlgo is an online visualization of algorithms and data structures website that presents visual images of algorithms and data structures. It provides step by step explanation and a quiz (VisuAlgo, n.d.). This works very well for demonstrating algorithms but still requires more advanced play-back technologies.
- This catalogue and repository of algorithm visualization tools and algorithm research materials developed is called AlgoViz. This primary advantage of it is organized and rather than a combined interactive dashboard (Shaffer et al., 2010).
- This PhET Interactive Simulations are a series of interactive science and math simulations designed on the concept of first-hand visual manipulation. This doesn't however cover computer science algorithms or games on the strategy board (University of Colorado Boulder, n.d.).
- Algorithm Visualizer is an algorithm-visualization tool that uses executable code examples (Algorithm Visualizer, n.d.). This provides useful information about coding however it is difficult for absolute beginners to use as it requires some knowledge of coding to use the information (Algorithm Visualizer, n.d.; Mason & Seton, 2026).

**Table 2.1: Comparison of existing educational platforms**

| Platform | Main focus | Main strengths | Identified limitations | Difference in the proposed platform |
|---|---|---|---|---|
| VisuAlgo (VisuAlgo, n.d.) | Algorithm and data-structure visualization | This provides animations, explanations and quizzes | Primarily demonstrates algorithms through prepared visualizations | This proposed platform requires learners to make manual decisions and also includes strategy games |
| AlgoViz (Shaffer et al., 2010) | Repository of algorithm-visualization resources | This provides access to different visualization tools and research materials | Functions mainly as a catalogue rather than one integrated learning environment | In this proposed platform provides five activities through one shared dashboard |
| PhET (University of Colorado Boulder, n.d.) | Interactive science and mathematics simulations | This supports visual experimentation and direct manipulation | Does not focus specifically on computing algorithms or strategy games | This proposed platform mainly focuses on computational-thinking activities |
| Algorithm Visualizer (Algorithm Visualizer, n.d.) | Code-based algorithm visualization | This connects executable code with algorithm behavior | May require programming knowledge that beginners do not yet have | In this proposed platform allows learners to practice selected concepts without writing code |

### 2.6.3 Research Gap and Project Response

Existing educational tools provide useful support for learning algorithms and programming concepts. However, prepared visualisations that offer limited learner interaction may not fully support conceptual understanding. Interactive features that require learners to predict, select or control algorithm steps can provide more active learning opportunities (Sibia et al., 2025). Code-oriented tools may also create an additional barrier for beginners because learners must understand programming keywords and syntax while exploring the underlying algorithm (Mason & Seton, 2026).

This comparison of existing platforms also shows that many tools focus on either algorithm visualization, programming exercises or subject-specific simulations. Among the platforms reviewed in this study, there is limited integration of active algorithmic activities and strategy-based games within one beginner-friendly environment. This represents the main gap addressed by the project.

This project responds to this gap by integrating Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess within a shared dashboard. Bubble Sort and Dijkstra's Pathfinding require learners to make selected algorithmic decisions. The three board games provide opportunities to practice rule-based reasoning, state evaluation and strategic planning. Learners can therefore explore these concepts without writing programming code.

This system was developed as a lightweight, browser-based prototype using HTML, CSS and JavaScript. Its static files are served locally through a Node.js server. A supported browser and Node.js are therefore required on the computer hosting the platform. The project is not intended to replace formal programming courses. It does not include a backend database, remote user accounts, learning-management-system integration or an automated adaptive-learning engine. These limitations define the current project scope and indicate possible areas for future development.

### 2.6.4 Project Originality

The originality of this project is that it offers two kinds of practice for computational thinking in one package, without being in a computational environment. Bubble Sort and Dijkstra's Pathfinding are examples of Learner Algorithmic Decisions, which need to be performed manually, while Tic-Tac-Toe, Three Men's Morris and Chess are examples of Legal Move Reasoning, State Evaluation & Strategic Planning. Among the platforms reviewed in this study, most focus on algorithm visualization, code-based activities or subject-specific simulations. None of the reviewed platforms combines manual algorithmic activities and strategy-based board games within one shared environment. This project brings these activities together through a common dashboard with shared navigation, instructions, feedback, sound and theme controls, and selected progress storage.

This project does not claim that all the individual algorithms and board games are new. The originality lies in the choice, the presentation and the integration of the five activities in a single light prototype learning for the beginners. This combination has not yet been tested in a user study or learning controlled evaluation for its impact on learning.

### 2.7 Summary

Recent research indicates that active participation, appropriate scaffolding and interactive visualisation can support the teaching of computational-thinking concepts. However, visualisation alone does not guarantee conceptual understanding. Learners benefit more when they can predict outcomes, make decisions and observe how their actions affect the algorithm's state (Hassan & Zilles, 2024; Sibia et al., 2025). The manual decisions required in Bubble Sort and Dijkstra's Pathfinding follow this interactive approach. Because these activities use visual controls rather than written code, learners can practise selected algorithmic reasoning without first managing programming keywords and syntax, which can create additional cognitive demands for beginners (Mason & Seton, 2026).

These principles informed the design of the five platform activities. Bubble Sort requires learners to make comparison-and-swap decisions, while Dijkstra's Pathfinding requires node-selection and distance-relaxation decisions. Tic-Tac-Toe, Three Men's Morris and Chess provide opportunities to practise legal-move identification, state evaluation and strategic planning. Instructions, highlighted elements and immediate feedback provide scaffolding throughout the activities. Browser localStorage consistently saves the supported scores, Dijkstra level progress, selected outcome statistics, theme preferences and help-display settings. Unfinished activity states are intentionally not stored.

Existing web-based tools demonstrate different approaches to algorithm and simulation-based learning. VisuAlgo provides prepared algorithm and data-structure visualisations, while Algorithm Visualizer connects executable code with algorithm behaviour (Halim, 2015; Algorithm Visualizer, n.d.). The platform developed in this project differs by combining two interactive algorithm activities and three strategy games within one shared dashboard. This integration is the project's design response to the limitations identified in the reviewed tools. It should not be treated as evidence that the platform improves computational-thinking ability, as this would require a formal empirical study using validated assessment instruments (Zhang et al., 2024). The literature reviewed in this chapter informed the methodology, system design and implementation presented in Chapters Three and Four.

# CHAPTER THREE

# METHODOLOGY AND SYSTEM DESIGN

## 3.1 Introduction

This chapter describes the development methodology, system architecture, user-interaction model, data-persistence strategy, and hardware and software requirements of the Multi-Game Learning Platform. The platform was designed as a lightweight, browser-based application using HTML, CSS, JavaScript, and a local Node.js server. It does not require cloud infrastructure, a backend database, or a user-account system.

The design was informed by research on computational thinking, interactive learning, game-based learning, and educational scaffolding. These areas emphasize active learner participation, clear guidance, visual feedback, and opportunities to practice algorithmic and logical reasoning (Videnovik et al., 2023; Sweller, 2024; Sibia et al., 2025). The following sections explain how these principles informed the development and organization of the platform.

## 3.2 Development Methodology

This project followed a modified Scrum methodology based on Agile principles. Scrum is a framework for iterative and incremental software development that organizes work into short, time-boxed periods known as sprints. Each sprint focuses on completing a selected group of tasks and producing a measurable improvement to the system. This approach allows completed work to be reviewed regularly and problems to be identified before the next sprint begins. It also supports Agile principles such as incremental delivery, continuous review, and adaptation to change (Lee & Chen, 2023; Sassa et al., 2023).

Because this was an individual project, Scrum was adapted to suit a single-developer environment. The researcher carried out the development responsibilities and managed the product backlog, sprint tasks, implementation, and progress reviews. The product backlog contained requirements for the dashboard, shared platform services, five learning games, testing, and project documentation. The core requirements were defined during the first sprint and remained generally stable throughout development. However, detailed requirements were reviewed and refined during later sprints when implementation or integration issues were identified. These requirements could be clarified, revised, or reprioritized before being returned to the product backlog.

Tasks were selected according to their priority and relationship with other platform components.

At the beginning of each sprint, the required tasks and expected outcomes were identified and added to the sprint backlog. During the sprint, the selected features were designed, developed, integrated, and checked against the project requirements. Progress was monitored by the researcher without formal team-based Daily Scrum meetings. At the end of each sprint, the completed work was reviewed, identified problems were recorded, and unfinished tasks were returned to the product backlog. Feedback from the project supervisor was also considered when planning later improvements.

The main project scope and core requirements were defined during the first sprint and remained generally stable throughout development. These included the five learning games and the shared platform features. However, the detailed requirements were not treated as completely fixed. They could be clarified, reprioritized, or revised when implementation and integration checks identified technical problems or inconsistencies. Any unfinished or revised requirement was returned to the product backlog and considered during a later sprint.

This adapted Scrum approach was suitable because the platform contains five separate learning games supported by shared functions such as navigation, instructions, scoring, sound, theme controls, and browser storage. Individual components could be developed and integrated during different sprints. This helped identify problems involving activity rules, interface behavior, visual feedback, and system integration. It also allowed earlier components to be revised when problems were discovered during later development.

### 3.3 Development Sprints

This process of development was organized in five sprints, starting from 1 August and ending on 10 September 2026. Each sprint must have a clear goal and produce a product that contributed to this platform.

Sprint 1 activities included identifying the functional requirements, scope this browser-based work and initial interface and system designs. The emphasis of Sprint 2 was on developing the shared platform services, including the Dashboard, Navigation, Instructions, Theme Controls, Sound and Browser Storage. Sprint 3 focused on getting the five activity modules to work: Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess. Sprint 4 involved linking the activity modules to the common dashboard and checks for integration/implementation. During Sprint 5, there was a platform wrap up and the project report and supporting documentation including known issues on platform implementation were documented.

Sprints weren't completely independent. Some items were re-instated in the backlog and amended post integration and/or implementation tests uncovered items that required further assessment. Completed work was checked at the end of each sprint and the work that was still to be carried out was taken into account for the next sprint. This process was based on Scrum methodology which was adapted as mentioned in section 3.2. Table 3.1 is a overview of the key work and deliverable of each sprint.

**Table 3.1: Scrum sprint activities and outcomes**

| Sprint | Sprint goal | Work completed | Sprint outcome |
|---|---|---|---|
| Sprint 1 | Define the project scope and requirements | Identified the five learning activities, learner interactions, functional requirements and browser-based system scope. Prepared the initial interface and system designs. | Approved project scope, requirements and design plan |
| Sprint 2 | Develop the shared platform features | Created the dashboard, activity navigation, instruction pop-ups, theme control, sound control and localStorage functions. | Working dashboard and shared services in app.js |
| Sprint 3 | Develop the learning activities | Implemented Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess with their main rules and interactions. | Five working activity modules |
| Sprint 4 | Integrate and check the platform | Connected the five activities to the dashboard and checked navigation, activity rules, visual feedback, scoring, sound and browser storage. Recorded identified implementation issues. | Integrated platform and implementation-check findings |
| Sprint 5 | Complete evaluation materials and documentation | Prepared the acceptance-test framework, revised the system diagrams, documented the implementation and completed the project report and appendices. | Final report, test-case framework, project schedule and running instructions |

## 3.4 System Architecture

This platform uses a client-side, browser-based architecture. This means that the main application functions are executed in the user's web browser rather than on a remote server. The interface, activity rules, AI calculations, scoring, visual feedback, and progress-management functions are implemented using HTML, CSS, and JavaScript.

A lightweight Node.js HTTP server serves the static HTML, CSS, JavaScript, and media files to the browser through port 3000. The server does not execute game logic, calculate AI moves, manage user accounts, or store learner progress in a database. These operations are handled within the browser. Selected scores, unlocked levels, theme preferences, and help settings are stored locally using the browser's localStorage.

As shown in Figure 3.1, the shared services in app.js manage functions such as navigation, instructions, theme controls, sound, and selected progress data. Each activity has a separate JavaScript file that manages its rules, interactions, and visual updates. This modular structure separates the shared platform services from the logic of the individual activities.

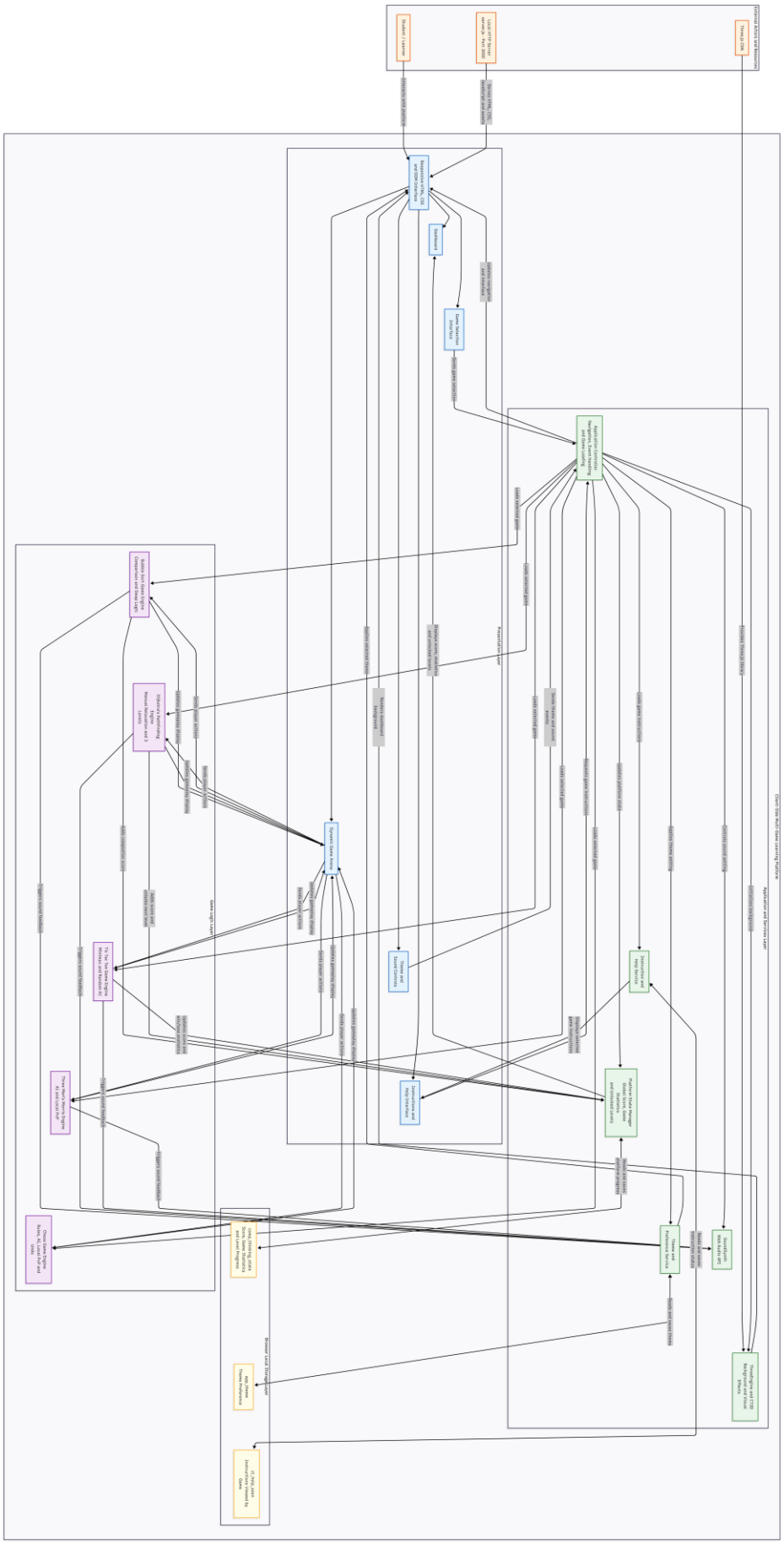


Figure 3.1: System Architecture Diagram

Files such as HTML, CSS, JavaScript, and media files are served from the local HTTP server. It does not execute any game logic, compute any AI moves, and it doesn't store any learner's progress. Scripts for the games are located in the games/ directory and loaded by index.html, with each game being initiated by selection, based on its place in the games/ directory. This is in contrast to only dynamically downloading a new game module at the time it is selected.

### 3.5 Behavioral Diagrams

The main role in the system is played by the student learner who interacts with the platform through desktop web browsers. The activities on the dashboard allow the learners to pick and choose from, instruction popups can be reviewed, algorithms can be manipulated manually and the audio feedback can be synthesized. A second actor, either a local human player (Player vs Player) or an algorithmic AI player exists in strategy games such as Chess, Tic-Tac-Toe and Three Men's Morris.

### 3.5.1 Use-Case Diagram

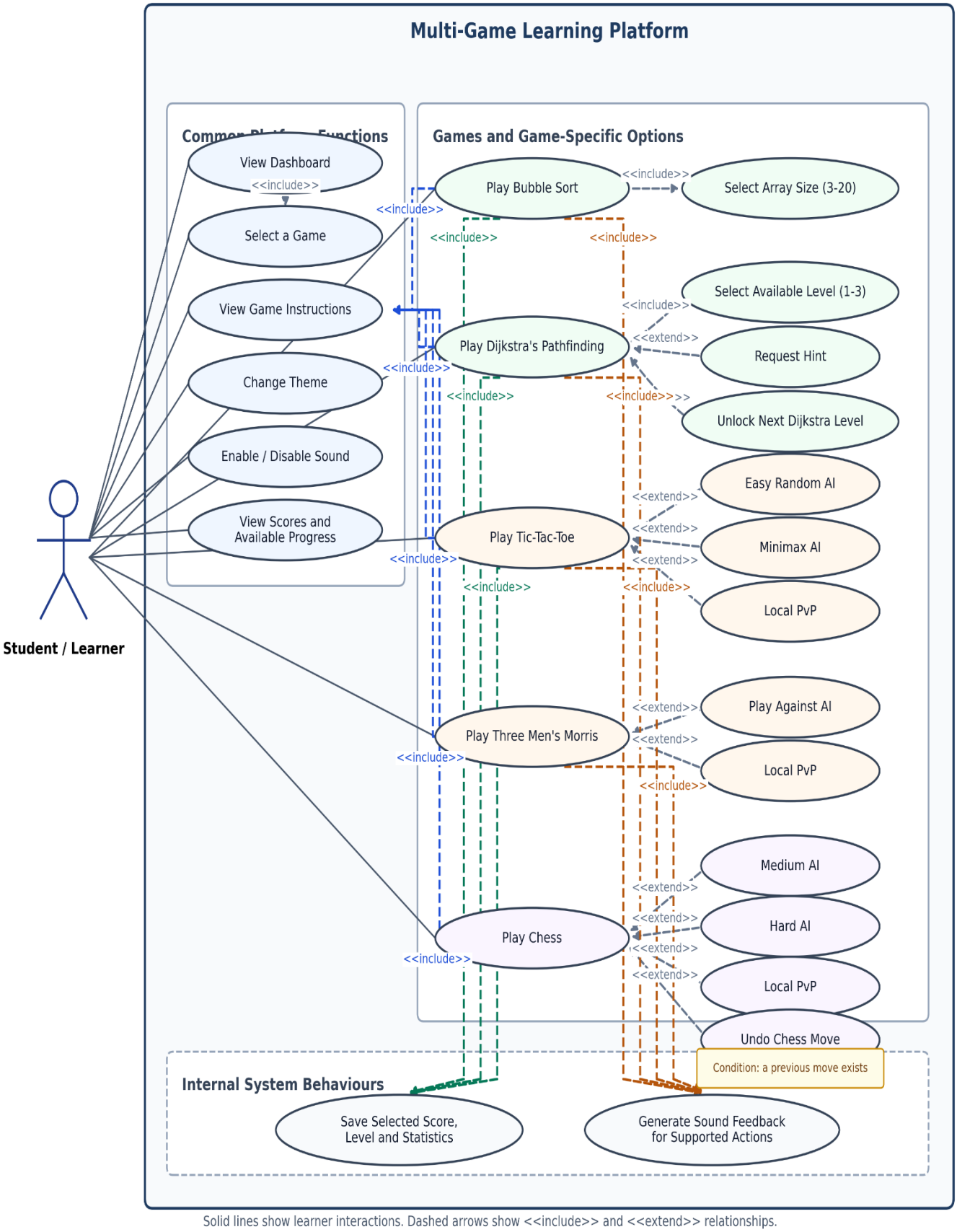



Figure 3.2: Use-Case Diagram

Figure 3.2 shows the main functions available to the learner in the Multi-Game Learning Platform. The learner can access the dashboard, select and play any of the five activities, view instructions, change the theme, control sound, and view supported progress. Each activity also provides its own options, such as array-size selection in Bubble Sort, levels and hints in Dijkstra's Pathfinding, and AI or local player modes in the strategy games.

The <<include>> relationship represents a function that is required as part of another activity. The <<extend>> relationship represents an optional or conditional function. For example, requesting a hint is optional in Dijkstra's pathfinding, while undoing a chess move is available only when a previous move exists. Features such as remote accounts, online multiplayer, cloud storage, and adaptive learning are not shown because they are outside the scope of the current project.

### 3.5.2 Activity Diagram

Figure 3.3 presents the main interaction flow of the Multi-Game Learning Platform. The process begins when the learner opens the platform in a supported browser. The application files are loaded, and saved preferences and supported progress data are retrieved from localStorage. The dashboard then displays the five available activity cards.

After selecting an activity, the system initializes the relevant module. Instructions may be displayed before the activity begins. If the instructions are shown, the learner closes them before proceeding. Otherwise, the learner can begin the activity directly.

During the activity interaction loop, the learner performs an action, such as selecting a node, entering a relaxation value, swapping array elements, or moving a game piece. The system checks the action against the rules of the selected activity. It then provides feedback, updates the activity state, and performs a computer move when required. This cycle continues until the activity is completed or the learner chooses to leave it.

When an activity is completed, the platform displays the result or completion feedback. If the activity supports saved progress, selected information such as scores, levels, or statistics is stored in localStorage. The learner can then return to the dashboard. If an unfinished activity is closed, its temporary state is not saved, and the learner also returns to the dashboard.

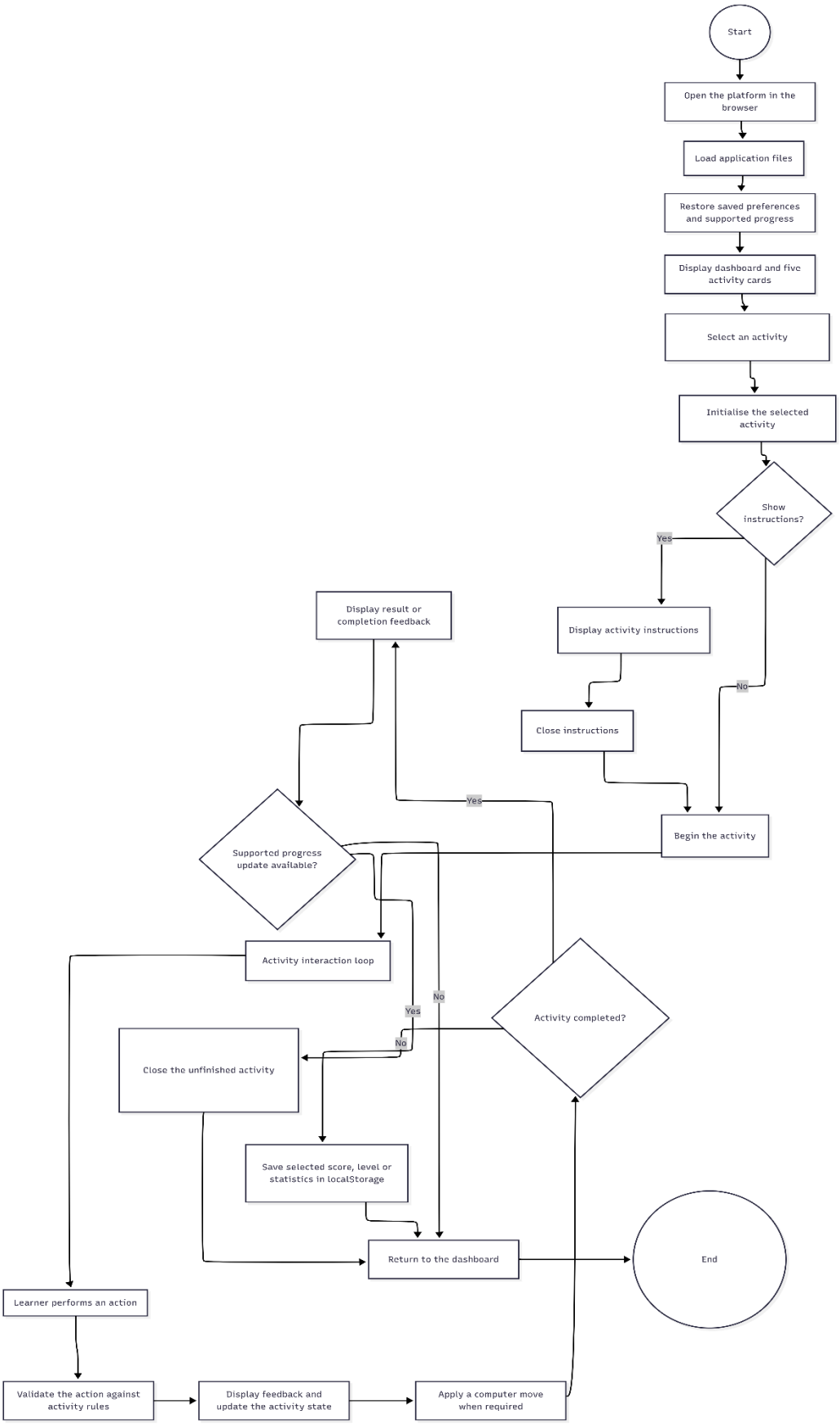



Figure 3.3 Activity Diagram

### 3.5.3 Class Diagram

The implementation class and module diagram presents the main structural components identified in the source code and the relationships between them. The system uses a mixed JavaScript structure that includes classes, shared objects, IIFE game modules, an HTML document, a Node.js server module, browser storage, and an external Three.js library.

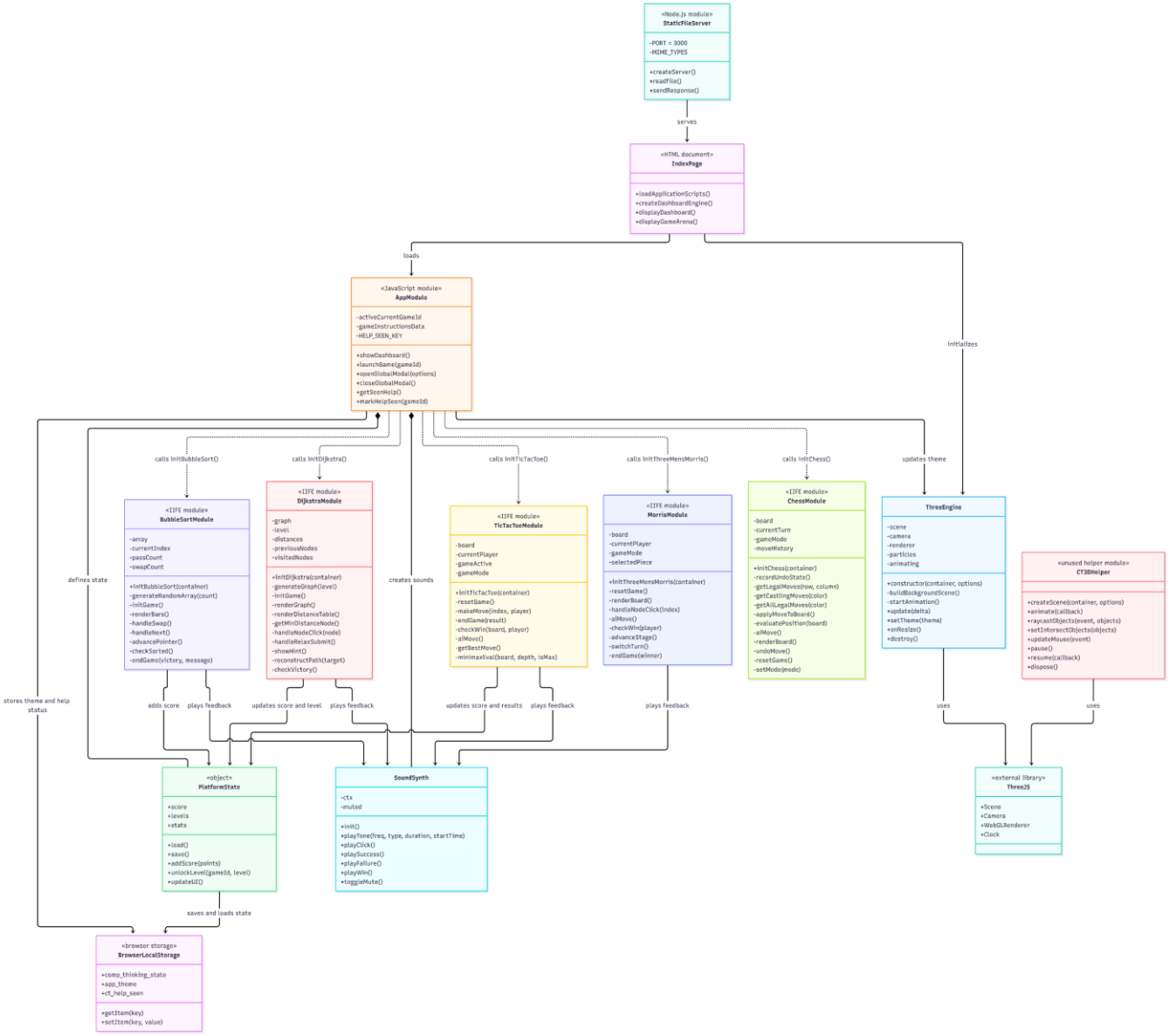



Figure 3.4: Class Diagram

Figure 3.4 shows the main classes, objects, and JavaScript modules identified in the source code. The application uses a mixed structure rather than a fully class-based design. SoundSynth and ThreeEngine are implemented as JavaScript classes, while the shared platform state is defined as an object. Each game is implemented as a separate IIFE module that exposes an initialization function through the window object.

The main application module controls navigation, instructions, activity loading, theme settings and shared sound functions. All five activity modules interact with the shared platform state when they update supported scores, unlocked levels or outcome statistics. Browser localStorage retains the supported platform state, theme preference and instruction-display status.

No inheritance relationship exists between the five game modules because they do not extend a common JavaScript class. The relationships in the diagram therefore represent actual dependencies and function calls found in the implementation.

### 3.5.4 Sequence Diagram

This sequence diagram shows how the main system components communicate when a learner opens the platform, selects an activity, performs an action, and returns to the dashboard. It presents the order of interactions between the learner, browser interface, local Node.js server, main application module, selected game module, shared sound service, platform state, and browser storage.

Figure 3.5 begins with the learner opening the platform through http://localhost:3000. The browser requests the application files from the local Node.js server. After the files are loaded, the application restores the supported platform state and theme preference from localStorage before displaying the dashboard.

When the learner selects an activity, the application opens the game arena and calls the initialization function of the selected game module. The application also checks whether the instructions for that activity have previously been displayed. On a first visit, the help status is recorded and the instructions open automatically.

During gameplay, the selected module validates the learner's action, updates its internal state, displays feedback and performs a computer move when required. Supported activities also generate sound feedback. When an activity is completed, its defined score, level or outcome statistics are updated through the shared platform state and saved in localStorage.

Finally, the learner can return to the dashboard. The application clears the current game arena and displays the dashboard again. Temporary activity state is not restored as part of this process.

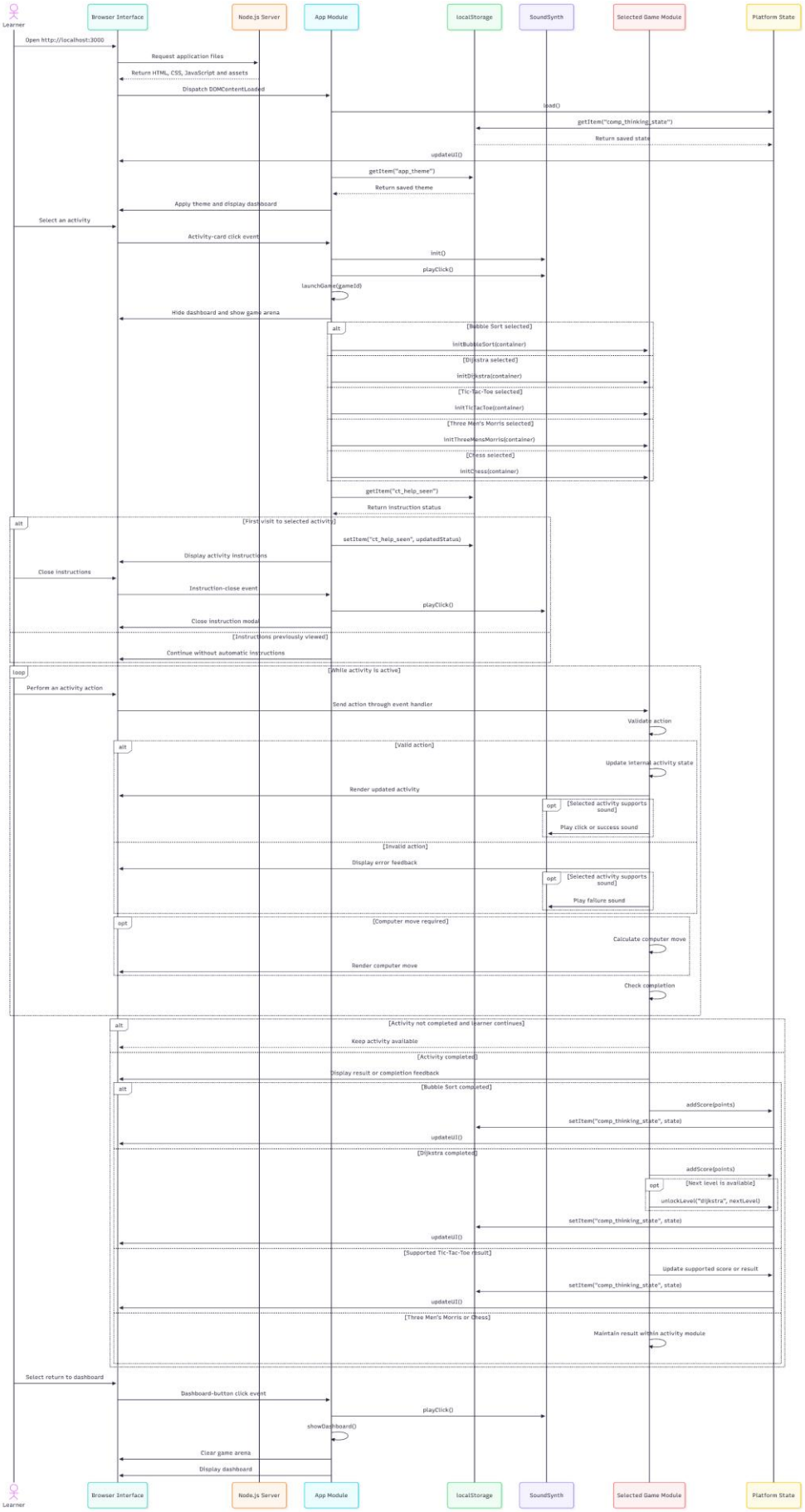



Figure 3.5: Sequence Diagram

**Table 3.2: Game modes and user controls**

| Game | Available choices | Boundary |
|---|---|---|
| Bubble Sort | The array size 3–20 shuffle, swap, advance | No selectable unlocked level sequence |
| Dijkstra | Three unlockable levels new graph hint enter relaxation values | Fixed curated map pools; no learner-adaptive difficulty |
| Tic-Tac-Toe | Easy random AI, Minimax AI, local PvP, restart | Evaluation display lists root-move scores |
| Three Men's Morris | AI or local PvP, select and move a piece, restart | No AI difficulty selector; stages are move-count labels |
| Chess | Local PvP Medium or Hard AI, board themes, promotion, undo | Depth-limited heuristic AI; no full recursive Minimax search |

### 3.6 System Requirements

Functional and Nonfunctional Requirements of Computational Thinking Multi-Game Platform are presented. The requirements were formulated depending on the primary objective of the project: to provide interactive activities to help beginners learn to practice computational thinking.

This system is realized as a prototype that is available over a web browser. It has 5 activities: Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess. It's made up of HTML, CSS, JavaScript, browser storage and the local node.js server.

### 3.6.1 Functional Requirements

This functional requirements describe the main behaviors that the platform is expected to provide. Some requirements were checked during development and system integration, while others were assessed through the functional test cases presented in Chapter Four. However, not all test cases were fully executed, and several implementation defects were identified. Browser storage is used to retain supported scores, Dijkstra level progress, theme preferences, and selected settings. Temporary information, such as unfinished boards, move histories, and active sessions, is generally not stored. Therefore, saved progress should not be described as complete game recovery.

**Table 3.3: Functional Requirements**

| ID | Functional Requirement |
|---|---|
| **FR1** | This system shall display all available games (Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris, and Chess) on the learner's dashboard upon opening the platform. |
| **FR2** | This system shall allow the learner to select a game from the dashboard and open it within the game arena. |
| **FR3** | This system shall verify the validity of all learner actions (e.g., sorting decisions, Dijkstra relaxation values, board moves, and chess moves) against the specific rules of the active game. |
| **FR4** | This system shall provide immediate, clear feedback to the learner following any action, utilizing mechanisms such as text messages, visual board/graph updates, status text, and audio cues. |
| **FR5** | This system shall calculate, display, and provide statistical analysis for the learner's score in every activity that includes a scoring component. |
| **FR6** | This system shall persistently store scores, levels, and statistical data to ensure no loss of data between user sessions. |
| **FR7** | This system shall carry over the learner's total score, selected level, and statistics if the learner logs out and returns within the same browser environment. |
| **FR8** | This system shall provide the learner with a functional reset or shuffle button at any time during an activity (e.g., shuffle for Bubble Sort, New Graph for Dijkstra, reset for board games). |
| **FR9** | This system shall allow the learner to select applicable game modes or difficulty levels prior to commencing a game (e.g., Tic-Tac-Toe modes, Chess AI levels, Dijkstra difficulty). |
| **FR10** | This system shall allow the learner to toggle common sound effects on or off during the session. |
| **FR11** | This system shall allow the learner to toggle between light and dark themes and persist that setting for the duration of the browser session. |
| **FR12** | This system shall display game instructions upon opening a game and allow the learner to recall these instructions at any point during the session via a help modal. |

The functional requirements show that the central features of the platform have been implemented and tested. Supported scores, levels and statistics are stored in browser storage. Unfinished boards, move histories and temporary session information are intentionally cleared after a reload, so saved progress should not be described as complete game recovery.

### 3.6.2 Non-Functional Requirements

Non-functional requirements outline the quality attributes of the system, like usability, performance, maintainability, and compatibility. Some of these requirements have been met by designing and implementing, others need to be tested in browsers and with the user.

**Table 3.4: Non-functional Requirements**

| ID | Non-Functional Requirement |
|---|---|
| **NFR1** | This system shall provide a consistent and easy-to-use interface. |
| **NFR2** | This system shall execute game actions without noticeable delays. |
| **NFR3** | This system shall operate correctly on commonly used modern web browsers. |
| **NFR4** | This system shall provide readable text, clearly distinguishable controls, game states, colours, and feedback elements. |
| **NFR5** | This system shall use a modular and organized file structure to facilitate future maintenance and modification. |
| **NFR6** | This system shall store learner progress in the browser's local storage and retain it between sessions on the same browser and origin. |
| **NFR7** | This system shall provide a responsive interface that adapts to different screen sizes and devices. |
| **NFR8** | This system shall operate without a complex backend database or user-account system and shall be served through a simple Node.js HTTP server. |
| **NFR9** | This system shall provide clear and appropriate feedback when a game module fails to load or a learner attempts an invalid action. |
| **NFR10** | This system shall support accessibility features, including theme switching, ARIA labels, and reduced-motion preferences. |

This non-functional requirements state how the system is supposed to be. The existing implementation has requirements oriented towards maintainability, local deplorability, responsive styling and shared navigation. Until they're tested, however, and documented, it's fair to say that performance, accessibility, mobile and cross browser is a target.

### 3.7 Hardware Requirements

It is a browser-based application and doesn't need specialized high-performance equipment. While Table 3.5 outlines the infrastructure needs, they may not always be the result of formal benchmarking but are practical suggestions for use when evaluating the system in a typical classroom or learning context.

The hardware requirements apply to the local computer used to run the Node.js server and access the Multi-Game Learning Platform through a web browser. This may be the developer's computer during development and testing, the evaluator's computer during demonstration, or the learner's computer during local use. The platform has not been deployed to a public or cloud server; therefore, separate production-server hardware requirements are not included.

**Table 3.5: Hardware Requirements**

| Component | Specification | Purpose |
|---|---|---|
| Central Processing Unit (CPU) | At least 800 MHz or higher | Processes all data and instructions within the computer system. |
| Computer | A desktop or laptop computer | Used to run the Node.js server and open the platform in a web browser. |
| Graphics card | GPU with at least 1 GB of RAM | Processes graphics and 3D visuals in the game without lagging the computer. |
| Memory | At least 4 GB of RAM | Provides sufficient RAM for the browser, local server and to accommodate game modules to run simultaneously without conflict. |
| Storage | Minimum of 100 MB free disk space | Disk space for the project files, Node.js dependency files, and any local data stored in the browser by the learner. |
| Display | A screen that has a resolution of at least 1366 × 768 pixels | A screen that can comfortably display the dashboard, game boards and visuals. |
| Graphics | Integrated graphics with basic browser support | Enough to render DOM elements, SVG, CSS animation and any content supported by Three.js. |
| Input devices | Keyboard and mouse or touch pad | Required to control the game, navigate the platform, and for any numerical or textual inputs. |
| Network access | Local network access to be able to run the server; internet connection is recommended | Local network access required for running the server over HTTP; internet connection is useful for external resources like Google Fonts, Three.js CDN etc. |

These specifications enable the game modules which rely primarily on DOM rendering, SVG rendering, and logic based on the browser itself to function with a high degree of efficiency with no need for dedicated graphics processing units (GPUs). In line with the aforementioned argument, the primary visualizer focuses on promoting an active decision-making by the participant and not on providing a complex animated system with high hardware requirements for the introduction to the visualizer. To make sure that JavaScript engines and SVG elements are displayed as intended for the user, correct browser compatibility is essential.

Core project files are downloaded efficiently through the local node server, but for optimal performance, an Internet connection is required since the project relies on remote Google fonts and proposed dependency on three.js CDN. These external resources should be available to users via their browser which should be a modern browser and JavaScript enabled.

### 3.8 Software Requirements

This software requirements apply to the local computer used to develop, run, test, and demonstrate the Multi-Game Learning Platform. Node.js and the project files are required to start the local HTTP server, while a supported web browser with JavaScript enabled is required to access the platform. Development tools, such as a source-code editor and browser developer tools, were used to create and inspect the system but are not required by a learner during normal use. Because the platform has not been publicly deployed, cloud-hosting and production-server software requirements are not included.

**Table 3.6: Software Requirements**

| Component | Technology / File | Usage |
|---|---|---|
| Interface | HTML / CSS / JavaScript | Handles the layout of the dashboard, the DOM-based controls and the SVG visualiser elements that are displayed on the dashboard. |
| Built-in modules of Node.js | http, fs, path | Serves project files as static content over HTTP on port 3000 via server.js. |
| Progress storage | Browser localStorage | Stores learner data like comp_thinking_state, app_theme, ct_help_seen etc. for use in successive sessions. |
| Sound engine | Web Audio API (SoundSynth) | Dynamically generated sound effects with oscillators (functions defined in app.js). |
| 3D infrastructure | Three.js 0.164.1 (r164) via CDN | 3D engine used as background by three-engine.js in index.html and imported via the import map in index.html. |
| Supporting 3D helper | ct-3d-helper.js (CT3D) | Helper file for 3D scenes that is not imported or used in index.html in the project at this time. |
| Typography | Google Fonts CSS import | Imports fonts from remote locations (Outfit and JetBrains Mono) to get consistent text styling throughout the platform. |
| Development tools | Text editor and browser DevTools | Used when developing the site for source editing, console debugging and manual feature verification. |

This application relies on local storage so there is only progress within one browser origin. If cookies and browsing data are removed from the site, its progress is lost; and there is no offline Service Worker implementation. Therefore, full offline capabilities cannot be ensured because of the importing of fonts and CDN scripts remotely.

### 3.9 Summary

In this chapter, methodology and system design to create the Multi-Game Learning Platform were proposed. The project was organized with five development sprints and using an adapted version of Scrum methodology. The sprints comprised of requirements and design, shared platform services, activity development, system integration, verification and documentation. This helped in the progressive development and review of each portion of the platform.

This client-side system architecture is explained in the chapter too. It is coded in HTML, CSS and JavaScript, with the application files residing on a small and simple nodejs server and served to the browser. Shared services in app.js control the navigation, theme settings, sound effects, and instructions and supported progress data in the app. Each board game (Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess) has its own JavaScript module to manage the rules and interactions. LocalStorage is used to store user settings (including selected settings) and progress information in the browser.

This structure and the flow of interactions of the platform was described using use-case diagram and activity diagram. The functional requirements outlined the behavior that the dashboard, learning activities, and shared controls were expected to have. The non-functional requirements consisted of usability, compatibility, responsiveness and accessibility and performance. The necessary hardware and software environment was also determined. These elements were combined to form the framework of design for the implementation/evaluation described in Chapter Four.

# CHAPTER FOUR

# IMPLEMENTATION AND EVALUATION

## 4.1 Implementation Overview

This Multi-Game Learning Platform consists of an entry page, a common stylesheet, common application logic, five activity files, a small node.js server and two files designed to facilitate the use of Three.js graphics. Each activity is wrapped inside an immediately invoked function expression, and gives access to an initialization function via the browser window object. App.js hides the dashboard area, opens the activity area and calls the respective initialization function when a learner chooses an activity.

The role of each of the main source files is listed in Table 4.1.

**Table 4.1: Source-file responsibilities**

| File | Responsibility |
|---|---|
| index.html | Defines the page structure, import map, activity-script references and Three.js background initialization. |
| styles.css | Provides the shared layout, color variables, themes, responsive rules and external font import. |
| app.js | Manages SoundSynth, platformState, dashboard navigation, activity launching, theme controls and help popups. |
| games/bubble-sort.js | Implements the interactive comparison, swapping and pointer-advancement activity. |
| games/dijkstra.js | Implements predefined graphs, manual distance relaxation, hints, level unlocking and score calculation. |
| games/tic-tac-toe.js | Implements the board rules, random AI, recursive Minimax, root-move score calculations, local player mode and outcome statistics. The candidate-move scores are displayed through the corrected Minimax evaluation panel. |
| games/three-mens-morris.js | Implements the nine-position movement game, adjacency checks and immediate-win or random computer behavior. |
| games/chess.js | Implements the Chess board, movement rules, heuristic computer play, move history, themes and undo controls. |
| three-engine.js | Implements the Three.js dashboard-background engine. It creates and manages the background scene used by the main platform interface. |
| games/ct-3d-helper.js | Contains additional Three.js scene utilities. It is not loaded by index.html or used by the five current activities. |
| server.js | Serves the static project files using built-in Node.js modules. |

This five game files must remain inside the games directory because this is the file structure referenced by index.html.

### 4.2 Algorithms and Data Structures

The key algorithms and data structures for the five games are described below. The pseudocode emphasizes the main decision-making operations and omits sound and visual-rendering operations unless they have a direct impact on the activity logic.

#### 4.2.1 Bubble Sort

The Bubble Sort game stores its values in a numeric array. It uses a comparison index, pass counter, swap counter and sorted boundary to track the sorting process. The learner can select an array size between 3 and 20 values. The generated values are displayed as vertical bars, with two adjacent bars highlighted during each comparison.

If the value on the left is greater than the value on the right, the learner must select Swap Elements. If the values are already in the correct order, the learner must select Advance Pointer. When an incorrect action is selected, the activity displays error feedback and keeps the current comparison unchanged.

**Algorithm 4.1: Bubble Sort Activity Process**

INPUT: Numeric array A

OUTPUT: Sorted array and updated score

```
BEGIN
    INITIALISE numeric array A

    SET index ← 0
    SET pass ← 1
    SET sortedBoundary ← LENGTH(A)
    SET swaps ← 0

    WHILE the activity is active DO
```

```
IF action = "SWAP" AND A[index] > A[index + 1] THEN
    SWAP A[index] WITH A[index + 1]
    INCREMENT swaps
    INCREMENT index

ELSE IF action = "ADVANCE"
        AND A[index] ≤ A[index + 1] THEN
    INCREMENT index

ELSE
    DISPLAY error feedback
    KEEP the current comparison
END IF

IF index reaches sortedBoundary − 1 THEN
    DECREMENT sortedBoundary
    SET index ← 0
    INCREMENT pass

    IF array A is sorted OR sortedBoundary ≤ 1 THEN
        FINISH the activity
        ADD 5 × LENGTH(A) to the shared score
    END IF
END IF
```

UPDATE the bars, pointer and status

END WHILE

END

The underlying comparison-and-swap procedure has a worst-case time complexity of $O(n^2)$, where $n$is the number of values in the array. This complexity describes the sorting logic rather than the performance of the browser interface.

This implementation checks whether the array is sorted after each pass. Bubble Sort does not use an unlockable level sequence. Instead, the learner controls the difficulty by selecting an array size between 3 and 20 values.

### 4.2.2 Tic-Tac-Toe

The Tic-Tac-Toe game represents the board as a one-dimensional array containing nine elements. Each element initially contains an empty string and is later assigned either X or O. The game checks eight possible winning combinations: three rows, three columns and two diagonals. Three game modes are available: Easy AI, Minimax AI and local player-versus-player mode.

In Easy AI mode, the computer randomly selects one of the available board positions. In Minimax mode, O represents the maximizing player and X represents the minimizing player. The algorithm recursively evaluates the available moves and chooses the move with the highest score for O. Alpha-beta pruning is not implemented.

**Algorithm 4.2: Tic-Tac-Toe Minimax Evaluation**

INPUT: Current board, search depth and player type

OUTPUT: Evaluation score for the current board state

FUNCTION MinimaxEvaluation(board, depth, isMaximizing)

IF O has won THEN

RETURN 10 − depth

```
END IF

IF X has won THEN
   RETURN depth − 10
END IF

IF the board is full THEN
   RETURN 0
END IF

IF isMaximizing = TRUE THEN
   SET bestScore ← −∞

   FOR each empty position DO
      PLACE O in the position
      SET score ← MinimaxEvaluation(
         board, depth + 1, FALSE
      )
      RESTORE the position to empty
      SET bestScore ← MAX(bestScore, score)
   END FOR

ELSE
   SET bestScore ← +∞
```

```
        FOR each empty position DO
            PLACE X in the position
            SET score ← MinimaxEvaluation(
                board, depth + 1, TRUE
            )
            RESTORE the position to empty
            SET bestScore ← MIN(bestScore, score)
        END FOR
    END IF

    RETURN bestScore
END FUNCTION
```

**Algorithm 4.3: Selecting the Best Minimax Move**

```
INPUT: Current Tic-Tac-Toe board
OUTPUT: Best available move for O

BEGIN
    SET bestScore ← −∞
    SET bestMove ← undefined

    FOR each empty position DO
        PLACE O in the position
        SET score ← MinimaxEvaluation(board, 0, FALSE)
```

```
        RESTORE the position to empty

        IF the evaluation panel exists THEN
            DISPLAY the position and its score
        END IF

        IF score > bestScore THEN
            SET bestScore ← score
            SET bestMove ← position
        END IF
    END FOR

    RETURN bestMove
END
```

This Minimax algorithm evaluates the available root moves and calculates a score for each candidate move. The corrected interface includes the required evaluation panel, which displays these candidate moves and their calculated scores. Functional testing confirmed that the computer selected a legal move and that the candidate-move scores were displayed correctly. The panel presents the evaluated root moves rather than every branch explored during the recursive search.

This game is connected to the shared scoring and statistics system. A draw against the Minimax AI awards 15 points. An X win awards 10 points in Easy AI mode and 30 points in Minimax or local player-versus-player mode. An X win also increases the shared wins value. An O win increases the shared losses value only when the learner is playing against an AI.

This implementation identifies wins, losses and draws and updates the corresponding shared statistics. Functional testing confirmed that a draw was displayed correctly, increased the shared ties value and applied the defined draw score. The tested win and loss outcomes also updated their corresponding shared values.

### 4.2.3 Dijkstra's Pathfinding

The Dijkstra game represents each graph as a JavaScript object. Every node stores its display coordinates and a neighbor's object that maps adjacent nodes to their edge weights. The tentative distances, predecessors and visited states are maintained in separate objects. The graphs are displayed using SVG and Document Object Model elements. All graph templates use fixed, non-negative edge weights.

The learner actively participates in the algorithm instead of only watching an automatic animation. At each stage, the learner must select the unvisited node with the smallest tentative distance. The learner must then calculate and enter the updated distance for each unvisited neighbour of the selected node. Incorrect selections and relaxation values are rejected, recorded as mistakes and presented again for another attempt.

**Algorithm 4.4: Interactive Dijkstra Pathfinding Process**

INPUT: Weighted graph G, source node and target node

OUTPUT: Shortest path, final distance and completion reward

```
BEGIN
  FOR each node in graph G DO
    SET distance[node] ← ∞
    SET predecessor[node] ← NULL
    SET visited[node] ← FALSE
  END FOR

  SET distance[source] ← 0
  SET mistakes ← 0
  SET hintUsed ← FALSE

  WHILE target has not been visited DO
```

```
FIND the unvisited node u with the smallest
finite tentative distance

REQUIRE the learner to select node u

IF the learner selects an incorrect node THEN
    INCREMENT mistakes
    DISPLAY error feedback
    REQUIRE another selection
ELSE
    FOR each unvisited neighbour v of node u DO
        SET candidate ← distance[u] + weight(u, v)
        SET expected ← MIN(distance[v], candidate)

        REQUIRE the learner to enter expected

        WHILE the entered value is incorrect DO
            INCREMENT mistakes
            DISPLAY error feedback
            REQUIRE another value
        END WHILE

        IF candidate < distance[v] THEN
            SET predecessor[v] ← u
```

```
            END IF

            SET distance[v] ← expected
        END FOR

        SET visited[u] ← TRUE
    END IF
END WHILE

RECONSTRUCT the shortest path by following
the predecessors from the target node

SET baseReward according to the selected level
SET penalty ← MIN(mistakes × 3, baseReward − 10)

IF a hint was used THEN
    SET bonus ← 0
ELSE
    SET bonus ← 10
END IF

SET reward ← MAX(baseReward − penalty + bonus, 10)

ADD reward to the shared score
```

IF another level is available THEN

UNLOCK the next level

END IF

DISPLAY the shortest path, distance, reward and time

END

The implementation finds the next node by scanning the keys of the graph object and selecting the unvisited node with the smallest finite distance. It does not use a priority queue. The queue value displayed in the interface represents the number of unvisited nodes that currently have finite tentative distances.

Because the implementation repeatedly scans the graph to find the next node, its time complexity is $O(V^2 + E)$, excluding learner-response time and interface-rendering time. Here, $V$represents the number of vertices and $E$represents the number of edges.

The activity contains three unlockable levels. Level 1 uses graph templates with

### 4.2.4 Three Men's Morris

The Three Men's Morris activity implements a modified, movement-only version of the game on a board containing nine interconnected positions. At the beginning of the game, the learner's three pieces are placed at positions 6, 7 and 8, while the opposing pieces are placed at positions 0, 1 and 2. Unlike the traditional version of Three Men's Morris, this implementation does not include a placement phase or a capture-and-removal phase.

During each turn, a player selects one of their pieces and moves it to an adjacent empty position. A move is accepted only when the selected destination is empty and directly connected to the original position according to the predefined adjacency map. The activity supports play against the computer and local player-versus-player mode.

**Algorithm 4.5: Three Men's Morris Movement Process**

INPUT: Selected game mode and player moves

OUTPUT: Updated board state and game result

```
BEGIN
    INITIALISE a board containing nine empty positions

    PLACE learner pieces P at positions [6, 7, 8]
    PLACE opposing pieces A at positions [0, 1, 2]

    SET currentTurn ← P
    SET moveCounter ← 0
    SET gameStage ← 1
    SET gameActive ← TRUE

    WHILE gameActive = TRUE DO
        REQUIRE the current player to select a piece
        REQUIRE the current player to select a destination

        IF the destination is empty
                AND adjacent to the selected position THEN

            MOVE the selected piece to the destination
            INCREMENT moveCounter

            IF an active winning line is completed THEN
                FINISH the game
                DISPLAY the winner
            ELSE
```

```
        UPDATE the stage label when required
        CHANGE the current turn
    END IF

ELSE
    DISPLAY invalid-move feedback
    KEEP the board unchanged
END IF

IF game mode = "AI"
        AND currentTurn = A
        AND gameActive = TRUE THEN

    COLLECT all legal adjacent moves for A

    IF an immediate winning move exists THEN
        SELECT the winning move
    ELSE
        SELECT a random legal move
    END IF

    APPLY the selected computer move
    INCREMENT moveCounter

    IF A completes an active winning line THEN
```

```
            FINISH the game
            DISPLAY the computer as the winner
        ELSE
            UPDATE the stage label when required
            SET currentTurn ← P
        END IF
    END IF
END WHILE
END
```

This board contains eight geometrically possible three-position lines: three horizontal lines, three vertical lines and two diagonal lines. In this modified movement-only version, the top row [0, 1, 2] and bottom row [6, 7, 8] are treated as the initial formations and are intentionally excluded from the winning combinations. The remaining six lines are the active winning lines used throughout the activity. This exclusion is part of the implemented game rules rather than an implementation defect.

Stage 2 begins when the move counter reaches four, while Stage 3 begins when it reaches ten. These stages change only the labels and feedback shown to the learner. They do not introduce new placement, capture or movement rules.

During the computer's turn, the module collects all legal adjacent moves and checks whether any of them would produce an immediate win. If a winning move is available, the computer selects it. Otherwise, it chooses a legal move randomly. The computer does not use Minimax, and the activity does not provide selectable AI difficulty levels.

This game supports computer and local player-versus-player modes and uses the shared sound system. When a game finishes, the endGame function displays the result, plays the appropriate sound and updates the defined shared score and outcome statistics. Functional testing confirmed valid adjacent moves, rejection of invalid moves, detection of the six active winning lines and successful shared-statistics updates.

### 4.2.5 Chess

This Chess game represents the board as an 8 × 8 two-dimensional array. Each occupied square contains a piece code, such as WP for a White pawn or BQ for a Black queen, while an empty square contains null. The module contains logic for generating piece movements, filtering out moves that leave the player's king in check, handling special moves, generating algebraic notation and checking game-ending conditions.

The implemented special-move logic includes castling, en passant and pawn promotion. The module also checks for checkmate, stalemate, insufficient material, the fifty-move rule and threefold repetition. Functional testing with prepared board states confirmed that the tested ordinary moves, special moves and game-ending conditions produced the expected board states and results.

This game provides local player-versus-player mode, Medium AI and Hard AI. The computer controls the Black pieces in both AI modes. The board-evaluation function assigns material values of 1 to pawns, 3 to knights and bishops, 5 to rooks and 9 to queens. It also applies a small positional adjustment when a pawn occupies one of the four central squares.

Medium AI evaluates the position resulting from each candidate Black move. Hard AI performs an additional look-ahead step by generating possible White replies and evaluating the resulting positions. Neither mode implements a complete recursive Minimax search or alpha-beta pruning.

**Algorithm 4.6: Chess Computer-Move Selection**

INPUT: Current board and selected AI mode

OUTPUT: Selected move for Black

BEGIN

GENERATE all legal moves for Black

SET bestMove ← NULL

SET bestValue ← −∞

FOR each legal Black move DO

```
        CLONE the current board
        APPLY the candidate move to the cloned board

SET value ← EvaluatePositionFromBlackPerspective(cloned board)

        IF the move checkmates White THEN
            ADD 1000 to value
        ELSE IF the move places White in check THEN
            ADD 2 to value
        END IF

        IF selected mode = "Hard AI" THEN
            GENERATE possible legal White replies

            SET replyValue ← +∞

            FOR each legal White reply DO
                CLONE the candidate board
                APPLY the White reply
                EVALUATE the resulting position
                RETAIN the minimum evaluation
            END FOR

            IF a reply evaluation exists THEN
                ADJUST value using the retained reply evaluation
```

```
            END IF
        END IF

        IF value > bestValue THEN
            SET bestValue ← value
            SET bestMove ← candidate move
        END IF
    END FOR

    APPLY bestMove to the actual board
    UPDATE the turn, history and game state
    CHECK for check, checkmate or draw
    RENDER the updated board
END
```

This AI generates candidate moves for the Black side through the implemented move-generation functions. The evaluation and move-selection logic was checked to ensure that moves are assessed from Black's perspective. Functional testing confirmed that both Medium AI and Hard AI generated and applied legal moves for Black using the corrected evaluation direction.

This game also provides an undo control. Before a move is applied, selected state information is stored in undoHistory, including the board, turn, last move, half-move clock, castling rights, en passant target, position history and full-move number. The undo handler restores these stored values and redraws the board.

Functional testing confirmed that the undo control restored the board, current turn, move history and relevant game-state values correctly in the tested first-move and consecutive-undo scenarios.

This Chess game updates its defined shared score and game statistics after a completed game. Functional testing confirmed the tested ordinary moves, special moves, game-ending conditions, AI modes, undo behaviour and shared-progress updates under the recorded conditions.

### 4.3 Shared Application Services

#### 4.3.1 Platform State and Persistence

This shared platform state is defined as an object literal in app.js. It manages the total score, unlocked level values and selected game statistics. When supported progress changes, the information is converted into JSON and stored in the browser's localStorage under the comp_thinking_state key. This allows the progress to remain available when the platform is reopened using the same browser and origin. The state is managed through a shared object rather than a separate PlatformState class.

The following save() method stores the supported shared information in localStorage:

**Listing 4.1: Saving the shared platform state**

```
save() {
    localStorage.setItem(
        'comp_thinking_state',
        JSON.stringify({
            score: this.score,
            levels: this.levels,
            stats: this.stats
        })
    );

    this.updateUI();
}
```

This matching load function retrieves and parses the stored JSON. If a parsing error occurs, it is caught and reported in the browser console. The method does not provide complete schema and

data-type validation, and the save function does not handle every possible storage restriction or quota failure.

This shared state does not include active boards, timers, move histories or other unfinished-session values. Reloading or reopening an activity therefore starts a new activity session rather than restoring the exact state of an unfinished attempt.

**Table 4.2: Shared progress updates by activity**

| Activity | Shared score update | Other persistent update |
|---|---|---|
| Bubble Sort | Adds 5 points for each element in the completed array. | Does not use unlockable levels. |
| Dijkstra's Pathfinding | Adds a level-based reward adjusted for mistakes and hint use. | Unlocks the next available level, up to Level 3. |
| Tic-Tac-Toe | Adds points for completed outcomes. A draw is recorded and increases the score by half a point. | Updates the relevant win, loss and draw statistics. |
| Three Men's Morris | Updates the shared score after a completed game. | Updates its defined outcome statistics. |
| Chess | Updates the shared score after a completed game. | Updates its defined game statistics. |

This shared progress system stores the supported scores, unlocked levels and outcome statistics for all five activities. These values are restored when the platform is reopened using the same browser and origin. Temporary activity data, such as unfinished boards, timers and move histories, is not stored.

### 4.3.2 SoundSynth and Preferences

This first time it is needed, SoundSynth sets up an AudioContext. With its playTone function, it can play on a customizable period of time by using an OscillatorNode and a GainNode. There are other sounds that provide for click, successful, failure and winning sounds using multiple tones.

These effects are played in the browser, without the need of prerecorded sound files. They need browser support and audio limitations; this might require a user action, such as a mouse click, before audio can begin. The muted state is just a memory and is restored to its normal state when pages are refreshed.

This key app_theme is used to store selected theme of the interface. The automatic help status of the individual activities is stored as a JSON object, using the key of ct_help_seen. This does not mean that the learner has read or understood the instructions, but does show that the automatic pop up has been triggered.

This platform doesn't have ThemeManager or InstructionManager classes. These tasks are managed by functions and event listeners in app.js and other background logic is written in index.html.

### 4.3.3 Three.js Background Infrastructure

This platform is built with Three.js, which is used to animate the background of the dashboard (if desired). The needed Three.js module is linked to in the import configuration of index.html and the functions to create and control the background scene are contained in three-engine.js. The background container will be passed to the required function to set the renderer to the corresponding part of the web page.

While loading background, test was conducted in local node.js server. It could do this without preventing users from being able to install or access the dashboard or the 5 learning activities. Furthermore, the background was synced to the "visible" interface even on a normal use of the platform.

Other scene helpers are in the separate file games/ct-3d-helper.js. However, it is not a requirement in the five activity modules in force. It does not impact the dashboard background and learning activities.

## 4.4 Responsive Interface and Guidance

This interface has been designed with light/dark theme in mind, using CSS custom properties. Flexbox, CSS Grid and media queries are used to adjust the page layout. Each activity file also creates its own UI elements and styles. When learners open an activity, the dashboard is not displayed, and the activity that was selected is displayed in the activity area.

These techniques provide the foundation for responsive behavior, but do not guarantee that this platform will work on every desktop, tablet or mobile device. Adaptability and responsiveness needs to be tested with defined browser, viewport and device.

Some controls have accessibility markup and the page checks reduced-motion mode setting of the target background engine with the browser. By themselves, these are not enough to achieve conformity to Web Content Accessibility Guidelines. There is still no user-flow evaluation for keyboard navigation, the screen-reader output and touch interaction and focus order.

Activity instructions were analyzed in comparison with the behavior that was observed during the five activities. The instructions for Three Men's Morris explain how the pieces move and the initial setup of the pieces. In addition, the Chess instructions describe correctly the computer-player

behavior implemented by the heuristic. During functional testing instructions opened and appropriate guidance was given for all five activities.

## 4.5 Testing and Evaluation

### 4.5.1 Testing Evidence and Scope

This Multi-Game Learning Platform has been tested in local browser environment to ensure the most important functions are working as expected. Tested items were server start-up, loading dashboard, selecting activities, navigating activities, instructions, activity rules, scoring, level progression, visual feedback, sounds and setting themes and browser based storage. Five activities are chosen for testing: bubble sort, path finding (Dijkstra's algorithm), tic-tac-toe, Three Men's Morris and Chess.

Clear preconditions and test data were provided for each functional test. The result of each test was compared with the expected result and a Pass or Fail was awarded. The outcomes were directly logged in the functional test-case table. A test was marked PASSED when the behaviour that was observed, matched the behaviour that was expected. Test was deemed a failure if the system did not achieve the required result.

This final functional-testing record contains 24 test cases. FT14 initially failed because the Tic-Tac-Toe Minimax candidate scores were not displayed. After the evaluation panel was added and connected to the existing Minimax output, FT14 was repeated successfully. All 24 test cases therefore have a final status of Pass.

### 4.5.2 Functional and Non-Functional Testing

This functional testing was based on testing the features of the platform based on its requirements. Testing covered platform start-up, navigation of the dashboard, instructions for doing activities, Bubble Sort comparisons and swaps, rules of the Dijkstra, rules of the Three Men's Morris, Chess rules, Chess AI behavior, sound controls and browser-storage functions.

Functional testing was conducted, and access to the five activities and the loading of the server and dashboard were successful. Normal navigation, instructions, activity controls, scoring, feedback, progress and temporary-session behaviour were also tested under the recorded conditions. The initial Tic-Tac-Toe Minimax display problem was corrected, and the repeated FT14 test confirmed that the candidate-move scores were shown in the evaluation panel.

Interface Responsiveness, Browser Compatibility, Usability, Accessibility & Performance were the non-functional. During development and testing, the platform was utilized in a supported desktop browser. A comprehensive formal accessibility audit, performance benchmark and cross

browser evaluation was not conducted, though. So, the results from the functional testing should not be taken as the evidence of full compliance to accessibility or similar performance in all browsers and devices.

### 4.5.3 Functional Test Cases

This functional test cases were derived from the features of the Multi-Game Learning Platform that were implemented and the project requirements. Each test case is uniquely identified and described with test preconditions, test data, as well as expected test result, actual test result and final test result. The results for the 24 functional test cases are given in table 4.3.

**Table 4.3: Functional test cases for the Multi-Game Learning Platform**

| Test case ID and title | Test preconditions | Test data | Expected result | Test result | Status |
|---|---|---|---|---|---|
| **FT01: Start the platform** | Node.js and a supported browser are available. The project files are arranged correctly. | Command: node server.js,URL: http://localhost:3000 | The server starts on port 3000, and the dashboard displays all five activity cards. | The server started successfully on port 3000. The dashboard and all five activity cards were displayed correctly. The optional Three.js background also loaded without an error. | **Pass** |
| **FT02: Select activities from the dashboard** | The platform is running, and the dashboard is displayed. | Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess cards | Each activity card opens its corresponding module. A missing script is handled without affecting the rest of the platform. | All five activity cards opened their corresponding modules. The missing-script condition was also tested and handled correctly without stopping the platform. | **Pass** |
| **FT03: Return to the dashboard** | One of the five activities is open. | Dashboard navigation control | The current activity closes, and the dashboard becomes visible. Reopening the activity starts a new session. | The activity closed correctly, the dashboard was displayed and the activity could be reopened as a new session. | **Pass** |
| **FT04: Display activity instructions** | The platform is running, and the instruction control is available in each activity. | Instructions for all five activities | The correct instructions open and accurately describe the implemented rules of each activity. | The instructions opened correctly and accurately described the implemented rules for all five activities. | **Pass** |
| **FT05: Select a Bubble Sort array size** | Bubble Sort is open, and no sorting session is active. | Array sizes from 3 to 20 | The activity displays the selected number of values. Supported array sizes remain within the specified range. | Each selected array size produced the correct number of values, including the minimum and maximum supported size. | **Pass** |
| **FT06: Perform a valid Bubble Sort swap** | Bubble Sort is open with a highlighted pair in descending order. | Left value greater than right value, action: Swap Elements | This two values exchange positions. The comparison advances, and the relevant score or swap count is updated. | The selected values exchanged positions correctly. This comparison advanced, and the score and swap information were updated. | **Pass** |
| **FT07: Advance a valid Bubble Sort comparison** | Bubble Sort is open with a highlighted pair already in ascending order. | Left value less than or equal to right value; action: Advance Pointer | The values remain in their positions, and the comparison moves to the next pair without recording an unnecessary swap. | The values remained unchanged, the pointer advanced to the next pair and no unnecessary swap was recorded. | **Pass** |
| **FT08: Handle an incorrect Bubble** | Bubble Sort is open with an unsorted array and an | One incorrect action followed by valid actions | The incorrect action is rejected, feedback is displayed and the current comparison | The incorrect action was rejected, and appropriate feedback was displayed. The | **Pass** |

| | | | | | |
|---|---|---|---|---|---|
| **Sort decision** | active comparison. | | remains unchanged. The activity continues after the learner selects the correct actions. | comparison remained unchanged until a valid action was selected. The learner then completed the sorting sequence successfully. | |
| **FT09: Control Dijkstra level availability** | Dijkstra's Pathfinding is open, and a known level value exists in browser storage. | Stored levels 1, 2 and 3; one locked-level selection | Only unlocked levels can be opened. Completing a level unlocks the next level and stores the updated value. Locked levels remain inaccessible until the required earlier level is completed. | Level 1 increased to Level 2 after successful completion. The updated level was stored correctly, and attempts to open locked levels were prevented until the required conditions were met. | **Pass** |
| **FT10: Select the minimum-distance Dijkstra node** | An active graph with visible tentative distances is displayed. | One correct minimum-distance node and one incorrect node | The correct node is accepted. The incorrect node is rejected with feedback and does not advance the algorithm. | The minimum-distance node was accepted. The incorrect node was rejected, feedback was displayed and the algorithm did not advance incorrectly. | **Pass** |
| **FT11: Perform Dijkstra distance relaxation** | The correct active node has been selected. | One correct relaxation value and one incorrect value | A correct value updates the tentative distance and previous node. An incorrect value is rejected without changing the graph. | The correct value updated the distance and previous-node information. The incorrect value was rejected without changing the graph state. | **Pass** |
| **FT12: Use Dijkstra hints and complete a level** | An unlocked Dijkstra level is active. | Hint request, valid graph decisions and page reload | The hint relates to the current step. Completion displays the shortest path, updates progress and restores the unlocked level after reloading. | The hint displayed relevant guidance. The shortest path was shown after completion, and the unlocked level was saved and restored after reloading. | **Pass** |
| **FT13: Play Tic-Tac-Toe against Easy AI** | Tic-Tac-Toe is open in Easy AI mode with an empty board. | Valid cells, one occupied cell and the reset control | Valid moves are accepted, the computer makes a legal move, occupied cells reject additional moves and reset clears the board. | Valid player moves were accepted, and the computer produced legal responses. Occupied cells rejected additional moves, and reset cleared the board. | **Pass** |
| **FT14: Display Tic-Tac-Toe Minimax evaluation** | Tic-Tac-Toe is open in Minimax mode. | Board states that require the computer to evaluate possible moves. | The computer selects a legal move using Minimax, and candidate-move scores are displayed to the learner. | The Minimax algorithm selected a legal move, and the candidate-move scores were displayed correctly in the evaluation panel. | **Pass** |
| **FT15: Play Tic-Tac-Toe in** | Tic-Tac-Toe is open, and local player- | Move sequences producing an X | Players alternate correctly. Wins and draws are identified, | Players alternated correctly. X wins, O wins and draws were | **Pass** |

| | | | | | |
|---|---|---|---|---|---|
| **local player mode** | versus-player mode is available. | win, O win and draw | and the related score and statistics are updated. | detected. A draw increased the score by half a point as intended. | |
| **FT16: Apply Three Men's Morris rules** | Three Men's Morris is open in computer or local player mode. | The valid adjacent move, invalid non-adjacent move, occupied position and valid winning line | The valid moves are accepted, invalid moves are rejected and winning lines permitted by the implemented rules are detected. | Valid adjacent moves were accepted, while invalid, non-adjacent and occupied-position moves were rejected. The six active winning lines were detected correctly. The top and bottom starting rows were excluded according to the modified game rules, and the shared statistics were updated correctly. | Pass |
| **FT17: Validate ordinary Chess moves** | Chess is open in local PvP, Medium AI or Hard AI mode. | Representative legal and illegal moves, including a move that exposes the king | Legal moves are accepted. Illegal moves and moves that leave the player's king in check are rejected. | This tested legal moves were accepted. Illegal moves and moves that exposed the player's king were rejected correctly in the available game modes. | **Pass** |
| **FT18: Validate special Chess rules** | Suitable Chess board states have been prepared for each rule. | Castling, en passant, promotion, check, checkmate, stalemate, insufficient material, fifty move rule and repetition | Each special move and ending condition produces the correct board state and game result. | The tested special moves and ending conditions produced the expected board changes and game-status results. | Pass |
| **FT19: Check Chess AI and undo behaviour** | Chess is open, and at least one legal move can be made. | Medium AI, Hard AI and local PvP modes; move history and undo controls | Computer moves are legal and benefit the Black side. Undo restores the board, turn and relevant game-state values correctly. | Medium AI and Hard AI generated and applied legal moves for Black using the corrected evaluation direction. The undo control restored the board, current turn, move history and relevant game-state values correctly in the tested scenarios. | Pass |
| **FT20: Change and save the** | The dashboard is open, and browser | Light theme, dark theme, app_theme | One action changes the theme, updates the icon and stores the selected theme. | The light and dark themes changed correctly. The icon and stored app_theme | **Pass** |

| | | | | | |
|---|---|---|---|---|---|
| **platform theme** | storage is available. | value and page reload | The theme is restored after reloading. | value were updated, and the selected theme was restored after reloading. | |
| **FT21: Use the sound controls** | Browser audio is permitted, and the sound control is visible. | Mute and enable actions; click, success, failure and win events | Supported actions generate sounds when enabled and remain silent when muted. | Supported events produced the correct sounds when audio was enabled. No sound was produced while muted, and the results were documented. | **Pass** |
| **FT22: Display instructions automatically** | The dashboard is open, and the ct_help_seen value can be cleared from browser storage. | Cleared ct_help_seen value and repeated visits to each activity | Instructions open automatically on the first visit. They remain manually accessible but do not open automatically on every later visit. | Instructions opened automatically on the first visit. They remained available through the instruction control and did not reopen automatically during later visits. | **Pass** |
| **FT23: Save shared progress in browser storage** | The platform is running in a supported browser, and localStorage is available. | Score changes, Dijkstra level update, activity statistics, malformed JSON and page reload | Supported progress values are saved and restored. Invalid stored data does not cause an unhandled application failure. | Scores, Dijkstra level progression and the defined outcome statistics for Tic-Tac-Toe, Three Men's Morris and Chess were saved and restored successfully after reloading. Invalid stored JSON was handled without an unhandled application failure. | Pass |
| **FT24: Reset temporary session data** | Each activity can be opened, and an unfinished session can be created. | Unfinished Bubble Sort, Dijkstra, Tic-Tac-Toe, Three Men's Morris and Chess sessions followed by page reload | Temporary game states are cleared after reloading, and each activity begins a new session. | The reload procedure was completed for all five activities. Temporary session states were cleared, and each activity started a new session. | **Pass** |

### 4.5.4 Proposed User-Evaluation Survey

A user-evaluation survey was created to aid in a future evaluation of the Multi-Game Learning Platform. It is directed towards the usability, the clarity of the interface, the instructions for the activities, the visual feedback and perceived usefulness of the five learning activities. This survey was not conducted as part of this project. Thus, the number of participants and percentage of responses, usability scores, and verified learning outcomes are not provided in this report.

This suggested evaluation would be for undergraduate students who have basic computer-literacy skills. Each of the participants will be given a brief introduction about the platform, its purpose and how it works prior to using it. Participants would then engage in the use of at least three activities. These should include at least one algorithm game (e.g. Bubble Sort) and one strategy game (e.g. 3 men's morris, tic-tac-toe or Chess).

Participants would be asked to complete the following tasks:

1. Open the dashboard and select an activity.
2. Read the instructions for the selected activity.
3. Complete a Bubble Sort comparison sequence.
4. Perform at least one Dijkstra relaxation step.
5. Play one available board game.
6. Change the platform theme.
7. Enable and disable the sound.
8. Return to the dashboard and view the recorded progress.

After completing the tasks, participants would respond to the statements in Table 4.4 using the following five-point Likert scale:

1 = Strongly Disagree
2 = Disagree
3 = Neutral
4 = Agree
5 = Strongly Agree

**Table 4.4: Proposed User-Evaluation Questionnaire**

| No. | Evaluation statement | Evaluation area |
|---|---|---|
| 1 | The dashboard made it easy to find and select an activity. | Navigation |
| 2 | The purpose of each activity was clear to me. | Content clarity |
| 3 | The activity instructions were easy to understand. | Guidance |
| 4 | The controls responded as I expected. | Interaction |
| 5 | The visual feedback helped me recognize correct and incorrect actions. | Feedback |
| 6 | The Bubble Sort activity made the comparison and swapping process easier to follow. | Algorithm visualization |
| 7 | The Dijkstra activity helped me follow node selection and distance relaxation. | Algorithm visualization |
| 8 | The board games provided useful practice in logical and strategic decision-making. | Logical reasoning |
| 9 | The score and level information was easy to understand. | Progress information |
| 10 | The light and dark themes made the interface comfortable to use. | Interface design |
| 11 | I could use the platform without frequent assistance. | Ease of use |
| 12 | I would use the platform again to practice computational-thinking activities. | Intention to reuse |

This following optional open-ended questions could also be included:

1. Which feature of the platform did you find most useful, and why?
2. What changes would improve the platform?

Future evaluation could have the results for each item on the questionnaires summarized by frequency, percentage, mean and standard deviation. Common themes could be identified for the open-ended responses, for example navigation, instructions, visual feedback, difficulty of activity, technical issues. These findings would tell us about participants' experiences and opinions, but not directly about whether or not the platform itself enhances the ability to think in a computational way.

In a future study, the SUS could be used as a stand-alone regular usability scale (Brooke, 1996). The ten original questionnaire items and scoring procedure should be used, and not changed for the project. The original item-level responses should be kept in the anonymized form, in order to be able to check the score that was calculated.

This report does not claim a usability score, participant-satisfaction result or improvement in learning because the proposed survey was not conducted. The survey is presented as a framework for future evaluation of the corrected platform.

### 4.6 Summary

This chapter presented how the Multi-Game Learning Platform has been implemented and tested. It provided an overview of the common dashboard, navigation, instructions, sound controls, theme selection, browser storage and the five learning activities. It also covered the various algorithms and data structures employed in BubbleSort, Dijkstra's Path Finding, Tic-Tac-Toe, Three Men's Morris and Chess.

There were 24 functional test cases. The testing covered platform start-up, activity selection, navigation, instructions, activity rules, scoring, feedback, game progression, theme selection, sound controls and browser-storage behaviour. After FT14 was corrected and repeated, all 24 test cases passed, giving a functional-test pass rate of 100%.

FT14 initially failed because the Tic-Tac-Toe Minimax algorithm selected legal moves but the interface did not show the candidate-move scores. The required evaluation panel was added and connected to the existing Minimax output. FT14 was then repeated and passed, confirming that the candidate-move scores were displayed correctly.

A User Evaluation Survey, intended to help in the future assessment of usability and clarity of interface and usefulness for learning, was also proposed. The survey was not carried out in the current project. Hence, no participant data, usability metrics or validated achievement in computational-thinking skills are claimed.

This functional testing indicated that the various functions of the main platform features behaved as intended during the conditions tested. User evaluation, accessibility tests and measurement of performance and wider cross browser tests could give greater confirmation of the platform's wider education suitability.

# CHAPTER FIVE

# CONCLUSION AND DISCUSSION

## 5.1 Overview

This project unites five CT activities in one browser. Explicit algorithm practice and board games that require decisions and choices about legal-moves. The work is described using an iterative approach to development, client-side design and a local HTTP server. This should be assessed in relation to the prototype used, and the evidence available for this prototype.

## 5.2 Project Practical Contribution

The most important practical contribution of this project is the creation of a prototype in the browser which integrates five computational-thinking activities in a single interface. Direct algorithm problems and board games, where pattern recognition, reasoning about valid moves, state evaluation and decision making are required, are included. Instead of reading explanations or watching algorithms being executed automatically, learners are asked to make decisions in a number of activities and answer feedback.

In each of their activities they make a contribution in a different way:

1. **Bubble Sort:** This game allows the learner to observe step by step the sorting process. Learners will have to make a choice whether to swap two consecutive values or push the comparison pointer forward. When an incorrect action is selected the feedback is given and the current comparison is not changed. There is no unlockable system for Bubble Sort, but the size of the array can be changed by the learner, ranging from 3 to 20 elements.
2. **Dijkstra's Pathfinding:** In this game the students have to select the node with the smallest tentative distance that has not been visited and update the tentative distances of the adjacent nodes. These values are entered by the learners themselves, so that the intermediate steps of the shortest-path algorithm can be seen by the learner. The only activity that has unlockable levels is Dijkstra's Pathfinding. It is not a general purpose graph editor, but rather is based on a collection of pre-defined graphs.
3. **Tic-Tac-Toe:** There are three modes in this game: Easy AI (Random), recursive Minimax and local player-versus-player. The Minimax algorithm calculates scores for the available root moves before selecting an action. The corrected interface displays these candidate-move scores in an evaluation panel, and FT14 confirmed that the panel functions correctly in the tested environment.
4. **Three Men's Morris:** This is a game that simplifies the traditional game and is played on nine linked positions, with movements. At the beginning of the game, pieces are all in place and the

player moves the pieces to the neighbouring empty squares. There are 6 winning combinations and it features computer and PvP modes. No piece-placement stage and piece capture/difficulty selection.

5. **Chess:** This game provides a larger rule-based environment. It implements legal-move generation, check filtering, castling, en passant, pawn promotion, draw conditions, heuristic computer play and undo. The available modes are Medium AI, Hard AI and local player-versus-player. Functional testing confirmed the tested special moves, ending conditions, AI behaviour and undo scenarios.

Access to the all five activities is through a common dashboard. Each activity is in a separate .js file and each activity has its own init function. When the learner selects an activity, app.js removes visibility of the dashboard, reveals the activity area and calls to the appropriate function. Modular structure which isolates logic of the various activities but has a common entry point and navigation structure.

The main technologies of the platform were HTML, CSS and JavaScript. The application files are served by a small http server with node.js, using node.js's built-in http, fs and path modules. Activity logic is executed in the browser, not the server. In the current prototype, no frontend framework and user-account service, backend database or server-side game processing is required. Even though Express is listed in the package manifest, the included implementation of server.js doesn't use it to handle HTTP requests.

There are a few common characteristics that promote the five activities. For sound effects supported by SoundSynth, the Web Audio API of the browser is used for audio feedback without having to prerecord the sounds. They also implemented light and dark themes, activity instructions and selected activity progress storage in localStorage.

The browser-storage model uses three localStorage keys:

- comp_thinking_state stores the shared score, stored level values, including Dijkstra progression, and selected game statistics.
- app_theme stores the selected light or dark theme.
- ct_help_seen records whether the automatic help instructions have already been displayed for each activity.

This sound production and advancement storing occurs on the basis of the platform. The mute preference will only be saved in memory and revert to its default setting upon reloading the page.

There is a unified scoring and progression system throughout the five activities. Upgraded common shared values in Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess. Functional testing showed that Tic-Tac-Toe draws are correctly recorded and the common statistics were correctly updated in Three Men's Morris and Chess.

There are no activity boards in Dijkstra timers, full move histories, Chess undo data or other unfinished-session data is stored. These are kept in the temporary memory as long as an activity is open. This will result in a new session being created every time the learner re-loads the page and not continuing with an unfinished activity at the point of departure.

**Table 5.1: Practical contributions and current implementation boundaries**

| Practical contribution | Implementation basis | Current outcome and boundary |
|---|---|---|
| Unified educational interface | Five activity initialization functions are accessed through a shared dashboard. | Implemented for the five activities. The platform does not include an instructor dashboard or user-account system. |
| Active sorting exercise | Learners decide whether to swap adjacent values or advance the comparison pointer. | Implemented with array-size selection. Bubble Sort does not have unlockable levels. |
| Active pathfinding exercise | Learners select nodes and enter Dijkstra relaxation values. | Implemented using predefined graphs arranged across three level groups. It is the only activity with unlockable levels and is not a general-purpose graph editor. |
| Adversarial-search example | Tic-Tac-Toe uses recursive Minimax and calculates candidate root-move scores. | The calculation is present without alpha-beta pruning. The evaluation panel displays the candidate root-move scores, but the complete recursive search tree is not shown. |
| Rule-based board games | Three Men's Morris and Chess check moves against their implemented rules. | Implemented within the supplied variants. Functional testing confirmed the intended Three Men's Morris rules and the tested Chess AI, special rules and undo behaviour. |
| Activity feedback | Visual messages, state changes, move validation and browser-generated sounds provide feedback. | Implemented through visual messages, state changes, move validation and browser-generated sounds; the documented functional tests passed. |
| Shared scoring and progress | The platform stores the shared score, Dijkstra level progression and selected statistics under comp_thinking_state. | Implemented across the five activities for their supported scores, Dijkstra levels and outcome statistics. Active unfinished sessions reset after reloading by design. |
| Theme preference | The selected theme is stored under app_theme. | Implemented and verified: one action changes the theme, updates the icon, stores app_theme and restores the selection after reloading. |
| Automatic help status | Per-activity help-display flags are stored under ct_help_seen. | Implemented and verified for first-visit automatic display and later manual access. The instructions match the tested activity rules. |
| Lightweight local deployment | A small Node.js server delivers the client-side application files. | Provided for local use. Node.js must be installed on the serving computer, and some external resources may require an internet connection. |
| Three-dimensional background support | Three.js-related files and an import map are included. | Implemented. The Three.js dashboard background initialized without error in the recorded functional test. |
| Beginner-oriented operation | Shared navigation, instructions and immediate feedback are provided. | Intended by the design. Ease of use and accessibility have not been confirmed through a completed user or accessibility evaluation. |

This project's practical value is that it brings together a number of types of computational reasoning under the same 'learning' roof on the client side. The prototype allows the students to try out algorithmic steps, observe the decision making of the computer and to cope with different types of rules. It therefore can be used as a foundation for classroom demonstrations, lab activities and further development.

A set of 24 functional test cases was recorded and executed on the platform, as presented in Section 4.5.3. After the Tic-Tac-Toe Minimax evaluation panel was corrected and FT14 was repeated, all 24 tests passed, resulting in a 100% functional-test pass rate. No detailed evaluation of the individual

non-functional conditions has been completed, and the proposed user-evaluation survey has not been conducted.

This project should therefore be considered a working prototype, and the recorded functional results validate only the behaviour observed under the stated test conditions. They do not establish general reliability, confirmed usability, accessibility compliance or improvement in computational-thinking ability. Although FT14 has been corrected and repeated successfully, the proposed non-functional and user evaluations should still be completed before wider classroom use.

### 5.3 Contribution to Social Business

This platform can make a technical contribution as well as be utilized for wider social and educational purposes. It has the closest relationship with the UN Sustainable Development Goal (SDG) 4 Quality Education (United Nations, n.d.) as it offers an alternative way for students to interact with basic computational thinking concepts. However, there was no classroom study or social-impact evaluation included in the project. It is important to note, however, that the relationships listed below cannot necessarily be measured outcomes, but rather they are potential contributions.

- **SDG 4 Quality Education:** Interactive activities which involve sorting, path finding, pattern recognition, valid-move reasoning and strategic decision making. May be used to supplement classroom explanations, demonstration and guided laboratory activities. However, the impact on learning and inclusion and access to education has yet to be quantified.
- **SDG 9 Industry, Innovation and Infrastructure:** This project has a relatively light weight client side design, based on HTML, CSS and JavaScript, with a small node.js server. As it stands, it has no need for a backend database or to be processed on a server. But it has not yet been officially 'tested for performance on legacy machines and low-speed Internet connections.
- **SDG 10 Reduced Inequalities:** This is a browser-based learning resource could be used to extend access, if computers and browsers are available, and technical support is available locally. The access should not be said to be equal or universal, however, as the platform still requires appropriate hardware, local serving environment and for some external resources are still required to be connected to the internet.

In essence this platform demonstrates what a small bit of educational software can achieve in the field of education and social issues in general. Its social value would only be measurable if assisted by classroom use, learner-access studies and educator feedback and proper documentation of the educational value.

## 5.4 Roles and Responsibilities

This project was an independent project of Md Shakib Mostafa. Hence, during the planning, interface design, software development and implementation of the game, testing and report preparation, the task was performed by a single person. The responsibilities found in Table 5.2 were one process of individual development but are depicted as separate responsibilities.

**Table 5.2: Roles and responsibilities**

| Role | Responsibility |
|---|---|
| **Project Manager** | Planned this project activities, organized the development schedule, monitored progress and ensured that the main project requirements were addressed. |
| **UI/UX Designer** | Designed the dashboard, page layout, navigation flow, game interfaces, colour themes and general presentation of the platform. |
| **Frontend Developer** | Developed and integrated the platform using HTML, CSS and JavaScript. This included implementing the dashboard, navigation, shared state, theme handling, instructions and audio features. |
| **Game Developer** | Developed and integrated the five educational game modules: Bubble Sort, Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess. This included implementing their rules, interactions, feedback mechanisms and computer-controlled behavior. |
| **System Integrator** | Connected the individual game modules to the common dashboard and integrated shared services such as platformState, SoundSynth, localStorage and the local Node.js server. |
| **Tester** | Inspected the source code, checked selected game rules and algorithmic behavior, tested browser-related functions and recorded the errors and limitations identified during development. |
| **Technical Writer** | Prepared the project report, described the system design and implementation, organized the tables and appendices and compiled the supporting references. |

Working on it as a solo project allowed the project to be developed in one direction from the initial concept, to the interface, to the game logic and to the final documentation. It also provided me with first-hand experience on a variety of software development activities including project management, UI/UX design, algorithmic design, systems integration and technical documentation.

However there were also constraints with individual development. Especially, testing and evaluation was carried out by the same individual who designed and developed the system. This decreased the chance for independent technical review and specialist assessment. Future development could be improved by incorporating feedback received from individual software testers and accessibility reviewers and education specialists to get more objective feedback on the platform and to test the platform in a wider range of scenarios.

## 5.5 Discussion

This project was designed to develop an online resource to aid in the beginner learner's practice of PT through interactive practice. The working prototype has Bubble Sort, and Dijkstra's Pathfinding, Tic-Tac-Toe, Three Men's Morris and Chess all under one roof! Various problems, such as sorting, shortest path computation, and pattern recognition, to valid move testing and state evaluation and decision making, are presented in each activity. They together offer several chances for students to experience computational reasoning.

The most algorithm learning related games are Bubble Sort and Dijkstra's Pathfinding. In Bubble Sort, students make comparisons between two values and choose to perform a swap or move the pointer to the next. If a wrong decision is made, the feedback will be displayed and the current comparison will be saved. This gives the learner time to think again about the values before they go on. The size of the array can also be altered, but Bubble Sort doesn't have an unlockable level system.

This Pathfinding of Dijkstra also involves active participation of the learner. The learner should pick the smallest tentative distance node that has not been visited and compute the relaxation values for all of its neighboring nodes. If these values are entered manually, the intermediate steps of the shortest-path process will be displayed, and the algorithm won't run automatically. Presence of three unlockable levels with hints and feedback on wrong answers in the activity. However, it has predefined graph templates, and does not enable learners to draw their own graphs. Also, the number of nodes in the queue as shown in the interface are the nodes that are reachable but not visited, which is not a true implementation of a priority queue.

These three board games demonstrate other aspects of computational thinking. Tic-Tac-Toe provides an Easy mode in which the computer moves randomly, a recursive Minimax mode and a local player-versus-player mode. In Minimax mode, the algorithm assigns scores to the available root moves before selecting an action. The corrected evaluation panel displays these candidate-move scores to the learner, and FT14 confirmed that the display functions correctly. The implementation does not show the complete recursive search tree and does not use alpha-beta pruning.

Three Men's Morris provides practice in adjacent movement, movement restrictions and pattern recognition. The project implements a simplified movement-only version in which all six pieces begin on the nine-position board. The top and bottom starting rows are deliberately excluded from winning-line checks, leaving six active winning combinations. This is an intentional rule of the implemented variant. The activity supports computer and local player-versus-player modes but does not include selectable AI difficulty.

Chess is the biggest game that's based on rules on the platform. Contains code paths for the generation of legal moves, filter checks, castling, en passant, pawn promotion, drawing and heuristic computer play/undo controls. It demonstrates the use of a larger set of rules and state for determining what actions are possible and to help in computer controlled decisions.

The Chess computer controls the Black pieces. Functional testing confirmed that the Medium and Hard AI modes generated legal moves using the corrected Black-side evaluation direction. Prepared board states were also used to verify the tested special moves, draw conditions and undo behaviour.

This modular approach of the platform proved to be helpful during development. All activities will be separate JavaScript files, with each one having its own start-up function in it and app.js handling the dashboard/navigation/shared services. All the activity logic is done in the browser, the files are served by a small node.js http server. This helps to keep the platform relatively light, and prevents requiring user accounts, a backend database or server-side game processing. The server computer must however have node installed. Other external resources, like google fonts and intended dependency Three.js may also require an internet connection.

Shared services provide a consistent experience across the activities. SoundSynth produces supported sound effects through the browser's Web Audio API. The interface also provides light and dark themes, activity instructions and progress storage through localStorage. The comp_thinking_state key stores the shared score, Dijkstra level progression and defined game statistics. The app_theme key stores the selected theme, while ct_help_seen records whether the automatic help instructions have been shown for each activity. The sound-mute setting remains in memory and returns to its default value after a page reload.

Functional testing confirmed the defined progress updates across all five activities. Bubble Sort and Dijkstra's Pathfinding updated their supported score and level values, while Tic-Tac-Toe, Three Men's Morris and Chess updated their defined outcome statistics. These values remained available after the platform was reloaded in the same browser and origin.

This platform does not save unfinished-session data, such as active boards, Dijkstra timers, complete move histories or Chess undo data. Reloading the page therefore starts a new activity session. The storage feature should be understood as persistent storage for selected scores, levels, statistics and preferences, rather than complete recovery of an unfinished activity.

It was observed during integration testing that the Three.js dashboard background loaded correctly, the dashboard theme changed correctly per action and the dashboard interface was usable. Activity directions were also aligned with the activities being performed. The previously missing Tic-Tac-Toe candidate-score display was added, and the repeated FT14 test confirmed that it functioned correctly.

Functional testing confirmed that one theme-control action changed the theme once, updated the displayed icon and saved the selected theme. The saved theme was restored correctly after the page was reloaded.

This activity instructions were reviewed against the implemented behaviour. The Three Men's Morris guidance describes the modified movement-only rules, while the Chess guidance describes its

heuristic computer modes rather than presenting them as Minimax. Functional testing confirmed that the instructions displayed correctly for all five activities.

This platform has been tested and the following dashboard, five activities and main shared functions are identified as being functional at typical use during the process of development. Selected implementation checks (e.g. shared storage, progression of the levels in Dijkstra, terminal states of Tic-Tac-Toe, winning lines of Three Men's Morris, direction of the evaluation in Chess, and initialisation of Three.js) were also explored with respect to specific behaviours.

All 24 functional tests are documented in Section 4.5.3 with their preconditions, test data, expected results, observed results and final statuses. After FT14 was corrected and repeated, all 24 tests passed. These results apply only to the functions tested in the recorded environment and do not replace a full non-functional evaluation across different browsers, devices and accessibility settings.

A user-evaluation survey was designed, but did not get used in the current project. Hence no conclusions on the usability of the site based on the participants or percentages of satisfaction or a proven System Usability Scale (SUS) have been included in this report. The influence of the platform in learning was not also measured. Without a proper study of learning, it cannot be said that the use of the activities leads to an enhancement of computational-thinking ability.

Future assessment needs to be separated for usability and learning outcomes. To explore usability, participant's observation could be conducted during the activity; to perform the proposed project specific questionnaire or another standard usability questionnaire could be used. Learning outcomes would need to involve suitable tasks before and after an activity including aspects related to the sorting decisions, shortest-path calculations, valid-move reasoning, and explaining algorithmic decisions. Future studies should preserve the original anonymous responses, dates of study, the method used to score, some information of consent, and observation notes in order to allow for the checking of the findings reported.

This practical objective of the project was achieved by consolidating five computational-thinking activities in a browser-based platform. The main dashboard, activities, shared controls, progress updates, Three.js background, instructions and Chess behaviour were tested under the recorded conditions. The Tic-Tac-Toe candidate-score display was corrected, and FT14 passed after regression testing. The platform should still be treated as an educational prototype that requires non-functional evaluation and further user research.

### 5.6 Project Limitations

Some restrictions in the platform that makes it not possible to extend the current use. Progress is stored locally in the browser (localStorage), and is unique to a browser and origin. Clearing the site data from the browser will remove the information from the storage record and new information will be placed in a new record when accessing the platform with a new browser, through a new port or from a new browser. It doesn't provide user accounts, learner identification, cloud sync or an instructor's dashboard.

Only certain progress information is stored in the storage system. Does not presave active game boards, Dijktra timers, full game history or Chess undo data etc. When the page is refreshed during an activity the learner will not be able to resume from the precise point at which they stopped. Reopening the activity will start a new session.

Progress monitoring and shared scoring are provided across all five activities. Functional testing confirmed that scores, Dijkstra level progression, Tic-Tac-Toe draws and the defined Three Men's Morris and Chess outcome statistics were updated and restored correctly. Unfinished activity states are intentionally cleared after a page reload.

Corrected or clarified content of certain activities. Dijkstra's Pathfinding is based on an a priori defined graph template and the learner is not able to create his graph. Three Men's Morris does not feature any placement phase, capture rules or AI difficulty settings. Bubble Sort does not have an unlockable level system but does give the learner the opportunity to alter the size of the array. These options are acceptable for the prototype version that is being produced, but they don't provide the user with much personalisation and advanced use.

Chess contains the largest rule set and the greatest amount of game-state information in the platform. Functional testing verified the tested ordinary moves, special moves, draw conditions, AI behaviour and undo scenarios. However, the computer players remain depth-limited heuristic opponents rather than a complete recursive Minimax implementation with alpha-beta pruning. Wider regression testing would still be useful when the Chess logic is extended.

It features computer opponent and same device player-vers-player modes, but lacks online multiplayer. If that function were to be added, then the player management, realtime communication, backend service all would have to be added. Reliable offline functionality has not been confirmed either, as external resources such as Google Fonts, and planned for Three.js dependency, may require an Internet connection.

An evaluation of browser compatibility, device performance and accessibility was not carried out but 24 functional test cases were documented in Section 4.5.3. Hence, keyboard navigation, screen-

reader output, screen focus, and color contrast along with touch interaction and responsiveness shall be examined as complete user flows under specific non-functional test conditions.

Finally, the user-evaluation survey was never performed and there was no learning study in a controlled environment. Within the scope of the project, a claim of confirmed usability or participant satisfaction and measured improvement in CT ability is therefore not made. These limitations should be corrected with a technical correction and formal documentation of a browser and accessibility test as well as by a proper future user evaluation.

## 5.7 Future Work

This existing platform has a number of areas for improvement. The Tic-Tac-Toe evaluation panel has now been added and FT14 has passed after regression testing. Further work should focus on non-functional and user evaluations before more extensive classroom use.

### 5.7.1 Improving the Current System

This recorded functional-test can be used to view the Three.js dashboard background, which is integrated and functional. It is not an exceptional functional defect but future improvements may fine-tune its appearance and/or its use of resources.

Functional testing confirmed that one theme-control action changes the theme once, updates the icon, stores the selected theme and restores it after reloading. A future refactor could consolidate the related handlers and synchronization logic in one location to improve maintainability without changing the verified behaviour.

This game instructions now describe the implemented behaviour, including the movement-only Three Men’s Morris rules and the heuristic Chess computer modes. Future maintenance should keep these instructions synchronized with any later rule or interface changes. Project, course and supervisor details in the interface footer should also be reviewed whenever the platform is prepared for a new academic submission.

Functional testing confirmed that the Chess computer selects legal moves for Black using the corrected Black-side evaluation direction. Undo behaviour and the tested ordinary moves, special moves and ending conditions also passed. Future work could deepen the computer search, strengthen its evaluation strategy and extend regression testing to a wider range of prepared board states.

### 5.7.2 Improving Progress Tracking

This current shared progress model stores supported scores, Dijkstra level progression and activity statistics correctly. A future version could optionally preserve unfinished sessions so that learners can

continue after refreshing the page. Such persistence would need to restore each activity's state safely, including boards, timers, move histories and pending computer actions.

This current localStorage model saves only selected scores, levels and statistics. A future version could save the active state of unfinished activities. For example, it could retain the current Bubble Sort array and comparison position, Dijkstra distances and timer, Tic-Tac-Toe board, Three Men's Morris board and complete Chess state. The Chess record would need to include the board, turn, move history, castling rights, en passant state, draw-related values and undo history. Pending computer moves and timers would also need to be restored or safely cancelled. This would allow learners to continue after refreshing the page or returning to the platform later.

User accounts and a database could also be introduced in a future version. This would allow learners to access their progress from different devices and could support teacher accounts, class reports and learner-performance records. However, these functions would require appropriate authentication, security, privacy and data-protection measures.

### 5.7.3 Adding More Learning Activities

This platform currently focuses on five games. Additional algorithm-based exercises could provide learners with a wider range of computational-thinking practice. Possible future activities include:

- Binary Search;
- Linear Search;
- Graph Coloring;
- Depth-First Search;
- Breadth-First Search;
- Dynamic Programming;
- Stack and Queue operations; and
- recursion-based problem-solving activities.

Dijkstra's Pathfinding could be expanded with more graph templates, additional difficulty levels and a simple graph editor. This would allow instructors or learners to create their own weighted graphs. Bubble Sort could also include more structured levels, additional sorting algorithms and comparison activities involving Bubble Sort, Selection Sort and Insertion Sort.

Three Men's Morris could be expanded to include the full placement stage and other traditional rules that are suitable for its learning purpose. Chess could be improved through a clearer difficulty system; better explanations of computer moves and a more reliable AI strategy.

### 5.7.4 Adaptive Difficulty and Feedback

This current platform does not contain a complete adaptive-difficulty system. A future version could adjust challenge levels according to learner performance. Factors such as the number of mistakes, completion time, hint use and previous scores could be considered when choosing the next activity or level.

Dijkstra's Pathfinding could provide smaller graphs for beginners and more complicated graphs for experienced learners. Bubble Sort could gradually increase the array size and introduce additional sorting methods. Feedback could also explain why an answer is incorrect rather than only indicating that an error has occurred. This may help learners understand the reasoning behind the correct decision.

Achievement badges, progress milestones and completion records could also be introduced. These features should be designed carefully so that they support learning rather than distract learners from the computational concepts presented by the activities.

### 5.7.5 Testing, Accessibility and Deployment

This existing 24-case table in Section 4.5.3 should be retained as the baseline regression record. FT14 should remain in the regression suite and be repeated whenever the Tic-Tac-Toe Minimax logic or evaluation panel is changed. New execution dates, environments, evidence and statuses should be added when tests are repeated. Broader regression testing should accompany future changes to navigation, storage, computer actions, Chess rules, themes, sound and session-reset behaviour.

Accessibility should also be improved and evaluated. This includes keyboard navigation, visible focus indicators, screen-reader support, color contrast, touch-friendly controls and reduced-motion behavior. The dashboard and every activity should be tested on representative mobile devices, tablets and desktop computers before the platform is used in a wider educational setting.

This project could also be developed as a Progressive Web Application. This would allow selected resources to be cached and could support more reliable offline operation in classrooms with limited internet access. External fonts and other dependencies would need to be stored or handled locally for complete offline use.

### 5.7.6 Future Educational Evaluation

A future study should evaluate usability and learning outcomes separately. The project-specific questionnaire proposed in Table 4.4 may be used to examine participants' perceptions of the platform. If the standard System Usability Scale is also used, it should be administered separately using its original ten items and established scoring procedure. Original anonymized item-level responses, study dates, consent information, observation notes and scoring records should be retained so that the reported findings can be checked.

To measure educational value, learners could complete suitable computational-thinking tasks before and after using the platform. These tasks could assess sorting decisions, shortest-path reasoning, legal-move identification and the ability to explain algorithmic choices. Where practical, a comparison group and a later retention assessment could be used to determine whether any observed changes are associated with the platform and whether they continue over time.

Feedback from learners and lecturers could also be collected through interviews, observations or open-ended questionnaire responses. This would help identify which activities are most useful, which instructions are unclear and which changes are needed for classroom use.

### 5.8 Recommendations

Before wider classroom use, FT14 and the other functional areas recorded as passed should be included in regression testing whenever related code is changed. These areas include Three.js initialization, theme handling, instructions, Chess AI and undo, shared statistics and session reset. The project, course and supervisor details displayed in the interface should also be checked against the approved FYP 2 submission before release.

This current shared progress functions record supported scores, levels and activity statistics. If learners are expected to resume unfinished activities after reloading, active boards, timers, move histories and other session values could be added to a carefully designed persistence model. The sound preference could also be stored so that the learner's selected setting remains active after a reload.

This 24 functional test cases in Section 4.5.3 should remain part of the regression suite. FT14 has passed after correction and should be repeated whenever the Minimax logic or evaluation panel changes. Other passed cases should also be repeated when related implementation changes are made. Separate non-functional evaluation should examine browser compatibility, responsive behaviour, performance, keyboard access, focus order, screen-reader output, colour contrast and touch interaction under stated test environments.

For teaching purposes, the platform should be used as a supplement to explanation and discussion rather than as a replacement for instruction. Learners should be encouraged to explain why a Bubble Sort action, Dijkstra relaxation value or board-game move is valid. This would allow instructors to connect the visible activity state with the computational-thinking concept being demonstrated. However, the platform should not be described as improving learners' understanding until this has been examined through an appropriate learning study.

If this proposed user-evaluation survey is conducted in the future, the original anonymized responses, survey dates, scoring procedures, consent information and observation notes should be

retained. This project-specific questionnaire in Table 4.4 may be used to examine participants' perceptions of the platform. If the System Usability Scale is also used, it should be administered separately using its original ten items and established scoring procedure. Any future report should clearly distinguish usability findings from evidence of improvement in learning.

A future educational evaluation should use suitable pre- and post-activity tasks to examine changes in areas such as sorting decisions, shortest-path calculations, valid-move reasoning and the explanation of algorithmic choices. A comparison condition and a later retention assessment could also be included where practical. These measures would provide stronger evidence of the platform's educational value than participants' opinions alone.

Finally, future development should be guided by verified learner and instructor needs rather than by adding features without evaluation. Possible additions include user accounts, cross-device progress, teacher reporting, additional graph templates, a simple graph editor, improved hints, online multiplayer and offline support. These extensions would require changes to the current client-side architecture and should be supported by suitable security, privacy, deployment and testing plans.

## 5.9 Conclusion

This project developed a browser-based prototype that brings together five computational-thinking activities within one interface. Bubble Sort and Dijkstra's Pathfinding involve learners directly in selected stages of an algorithm, while Tic-Tac-Toe, Three Men's Morris and Chess introduce pattern recognition, valid-move reasoning, state evaluation and strategic decision-making. The platform is intended to support learning by allowing learners to interact with the activities and receive immediate feedback.

This prototype was developed mainly using HTML, CSS and JavaScript, supported by a small Node.js HTTP server that delivers the application files. Each activity is implemented in a separate JavaScript file and accessed through a shared dashboard. The platform also includes sound generated through the Web Audio API, light and dark themes, activity instructions and selected progress data stored through localStorage.

This platform was run and checked throughout development. Its local server, dashboard, five activities and main shared controls were found to operate during normal use. Learners can open the platform, select an activity, interact with its controls and return to the dashboard. These development checks demonstrate that the project achieved its main aim of producing a functional browser-based prototype.

This recorded functional testing initially identified a limitation in the Tic-Tac-Toe Minimax display. This required evaluation panel was added and connected to the existing candidate-score output, and FT14 passed when it was repeated. The other documented functional areas also retained their recorded Pass statuses. Broader browser, performance, accessibility and user evaluation remains necessary.

Section 4.5.3 records 24 functional test cases. After the FT14 correction and regression test, all 24 cases passed, giving a functional-test pass rate of 100%. The results support the stated behaviour under the recorded test conditions. The proposed non-functional conditions should still be assessed to provide stronger evidence of browser compatibility, responsive behaviour, accessibility, performance and touch interaction.

No user survey or controlled learning study was conducted during the current project. Therefore, this report does not claim confirmed usability performance, verified accessibility quality or a demonstrated improvement in learners' computational-thinking ability. These outcomes would need to be examined separately through the proposed questionnaire, an appropriately administered usability instrument and suitable pre- and post-activity learning assessments.

Overall, this project provides a practical foundation for presenting several forms of computational reasoning within a shared browser environment. Its main contribution is the integration of interactive algorithm exercises and rule-based strategy games within one educational prototype. With FT14 corrected and passed, the proposed non-functional, accessibility and user evaluations remain the next steps for strengthening its use in classroom demonstrations, guided practice and future educational research.

# REFERENCES


Algorithm Visualizer. (n.d.). Algorithm Visualizer. https://algorithm-visualizer.org/

Arslantaş, T. K. (2024). Theoretical framework for integrating computational thinking in education. In M. Saritepeci & H. Yildiz Durak (Eds.), Integrating computational thinking through design-based learning (pp. 15–31). Springer Nature Singapore. https://doi.org/10.1007/978-981-96-0853-9_2

Brooke, J. (1996). SUS: A 'quick and dirty' usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), Usability evaluation in industry (pp. 189–194). Taylor & Francis. https://digital.ahrq.gov/sites/default/files/docs/survey/systemusabilityscale%2528sus%2529_comp%255B1%255D.pdf

Chen, P., Yang, D., Metwally, A. H. S., Lavonen, J., & Wang, X. (2023). Fostering computational thinking through unplugged activities: A systematic literature review and meta-analysis. International Journal of STEM Education, 10, Article 47. https://doi.org/10.1186/s40594-023-00434-7

Espinal, A., Vieira, C., & Magana, A. J. (2024). Professional development in computational thinking: A systematic literature review. ACM Transactions on Computing Education, 24(2), 1–24. https://doi.org/10.1145/3648477

Grizioti, M. (2025). Computational thinking beyond coding: Exploring student computational practices while playing and modifying a socio-scientific simulation game with integrated computational tools. Educational Technology Research and Development, 73(4), 2251–2278. https://doi.org/10.1007/s11423-025-10477-y

Halim, S. (2015). VisuAlgo: Visualising data structures and algorithms through animation. Olympiads in Informatics, 9, 243–245. https://doi.org/10.15388/ioi.2015.20

Hamidi, A., Mirijamdotter, A., & Milrad, M. (2023). A complementary view to computational thinking and its interplay with systems thinking. Education Sciences, 13(2), Article 201. https://doi.org/10.3390/educsci13020201

Hassan, M., & Zilles, C. (2024). Evaluating algorithm visualizations, debuggers, and execution toward helping students understand code. In Proceedings of the 2024 ACM Virtual Global Computing Education Conference, Volume 2 (pp. 320–321). ACM. https://doi.org/10.1145/3649409.3691095

Hayatpur, D., Xia, H., & Wigdor, D. (2023). CrossCode: Multi-level visualization of program execution. In Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems. Association for Computing Machinery. https://doi.org/10.1145/3544548.3581390

Hui, H. B., & Mahmud, M. S. (2023). Influence of game-based learning in mathematics education on students' cognitive and affective domains: A systematic review. Frontiers in Psychology, 14, Article 1105806. https://doi.org/10.3389/fpsyg.2023.1105806

Kong, S.-C., Lai, M., Li, Y., & Chan, T.-Y. D. (2025). Co-development of senior primary students' computational thinking concepts and practices: Implications for teaching and learning. Education and Information Technologies, 30(3), 2859–2886. https://doi.org/10.1007/s10639-024-12922-6

Lee, H.-Y., Wu, T.-T., Lin, C.-J., Wang, W.-S., & Huang, Y.-M. (2024). Integrating computational thinking into scaffolding learning: An innovative approach to enhance science, technology, engineering, and mathematics hands-on learning. Journal of Educational Computing Research, 62(2), 431–467. https://doi.org/10.1177/07356331231211916

Lee, W.-T., & Chen, C.-H. (2023). Agile software development and reuse approach with Scrum and software product line engineering. Electronics, 12(15), Article 3291. https://doi.org/10.3390/electronics12153291

Li, M., Ma, S., & Shi, Y. (2023). Examining the effectiveness of gamification as a tool promoting teaching and learning in educational settings: A meta-analysis. Frontiers in Psychology, 14, Article 1253549. https://doi.org/10.3389/fpsyg.2023.1253549

Liu, Z., Gearty, Z., Richard, E., Orrill, C. H., Kayumova, S., & Balasubramanian, R. (2024). Bringing computational thinking into classrooms: A systematic review on supporting teachers in integrating computational thinking into K–12 classrooms. International Journal of STEM Education, 11, Article 51. https://doi.org/10.1186/s40594-024-00510-6

López-Fernández, D., Gordillo, A., Pérez, J., & Tovar, E. (2023). Learning and motivational impact of game-based learning: Comparing face-to-face and online formats on computer science education. IEEE Transactions on Education, 66(4), 360–368. https://doi.org/10.1109/TE.2023.3241099

Mason, R., & Seton, C. (2026). The hidden burden of keywords: Cognitive load and language differences in novice Python programming. Education Sciences, 16(4), Article 657. https://doi.org/10.3390/educsci16040657

Matthew, G., Pelser-Carstens, V., Bunt, B., & Bunt, L. (2025). Enhancing student engagement and knowledge retention through game-based learning: A comprehensive framework integrating game design and learning theories. European Conference on Games Based Learning, 19(2), 591–599. https://doi.org/10.34190/ecgbl.19.2.3873

Mikrouli, P., Tzafilkou, K., & Protogeros, N. (2024). Applications and learning outcomes of game-based learning in education. International Educational Review, 2(1), 25–54. https://doi.org/10.58693/ier.212

Montuori, C., Gambarota, F., Altoè, G., & Arfé, B. (2024). The cognitive effects of computational thinking: A systematic review and meta-analytic study. Computers & Education, 210, Article 104961. https://doi.org/10.1016/j.compedu.2023.104961

Mukasheva, M., Kalkabayeva, Z., & Pussyrmanov, N. (2023). Visualization of sorting algorithms in the virtual reality environment. Frontiers in Education, 8, Article 1195200. https://doi.org/10.3389/feduc.2023.1195200

Palop, B., Díaz, I., Rodríguez-Muñiz, L. J., & Santaengracia, J. J. (2025). Redefining computational thinking: A holistic framework and its implications for K–12 education. Education and Information Technologies, 30, 13385–13410. https://doi.org/10.1007/s10639-024-13297-4

Sassa, A. C., Almeida, I. A. de, Pereira, T. N. F., & Oliveira, M. S. de. (2023). Scrum: A systematic literature review. International Journal of Advanced Computer Science and Applications, 14(4). https://doi.org/10.14569/IJACSA.2023.0140420

Shaffer, C. A., Cooper, M. L., Alon, A. J. D., Akbar, M., Stewart, M., Ponce, S., & Edwards, S. H. (2010). Algorithm visualization: The state of the field. ACM Transactions on Computing Education, 10(3), Article 9. https://doi.org/10.1145/1821996.1821997

Sibia, N., Liut, M., & Nobre, C. (2025). Exploring the role of visualization tools in enhancing computing education: A systematic literature review. In EuroVis 2025: Education Papers. The Eurographics Association. https://doi.org/10.2312/eved.20251027

Sweller, J. (2024). Cognitive load theory and individual differences. Learning and Individual Differences, 110, Article 102423. https://doi.org/10.1016/j.lindif.2024.102423

Tikva, C., & Tambouris, E. (2021). Mapping computational thinking through programming in K–12 education: A conceptual model based on a systematic literature review. Computers & Education, 162, Article 104083. https://doi.org/10.1016/j.compedu.2020.104083

United Nations. (n.d.). Goal 4: Quality education. https://www.un.org/sustainabledevelopment/education/

University of Colorado Boulder. (n.d.). What is PhET? PhET Interactive Simulations. https://phet.colorado.edu/en/about

Videnovik, M., Vold, T., Kiønig, L., Madevska Bogdanova, A., & Trajkovik, V. (2023). Game-based learning in computer science education: A scoping literature review. International Journal of STEM Education, 10, Article 54. https://doi.org/10.1186/s40594-023-00447-2

VisuAlgo. (n.d.). Visualising data structures and algorithms through animation. https://visualgo.net/en

Wu, T.-T., Asmara, A., Huang, Y.-M., & Hapsari, I. P. (2024). Identification of problem-solving techniques in computational thinking studies: Systematic literature review. SAGE Open, 14(2). https://doi.org/10.1177/21582440241249897

Yeni, S., Grgurina, N., Saeli, M., Hermans, F., Tolboom, J., & Barendsen, E. (2024). Interdisciplinary integration of computational thinking in K–12 education: A systematic review. Informatics in Education. https://doi.org/10.15388/infedu.2024.08

Yu, Q., Yu, K., & Li, B. (2024). Can gamification enhance online learning? Evidence from a meta-analysis. Education and Information Technologies, 29(4), 4055–4083. https://doi.org/10.1007/s10639-023-11977-1

Zhang, X., Aivaloglou, F., & Specht, M. (2024). A systematic umbrella review on computational thinking assessment in higher education. European Journal of STEM Education, 9(1), Article 2. https://doi.org/10.20897/ejsteme/14175

# Appendix A: Project Schedule

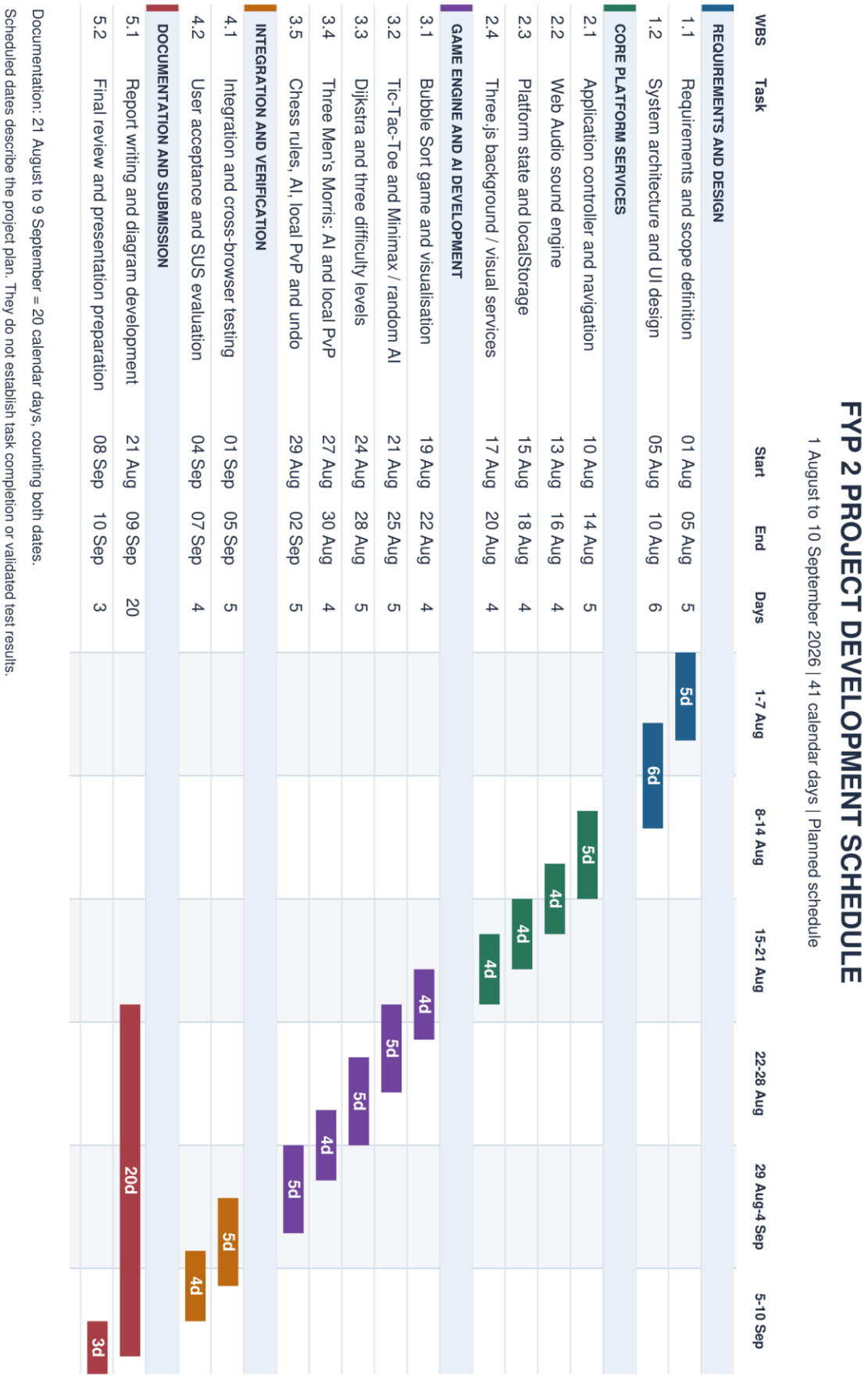



Figure A.1: Planned development and documentation schedule

# APPENDIX B: RUNNING THE PLATFORM

This appendix provides instructions on how to set up and operate the Multi-Game Learning Platform locally. The instructions are based on the file paths referenced by index.html and the provided Node.js server.

## B.1 Expected Folder Structure

The files required by the current implementation should be organized as follows:

```
project-root/
├── index.html
├── styles.css
├── app.js
├── server.js
├── three-engine.js
└── games/
    ├── bubble-sort.js
    ├── dijkstra.js
    ├── tic-tac-toe.js
    ├── three-mens-morris.js
    └── chess.js
```

This five game files must be placed within the games folder as index.html loads them from the games folder. The associated activity will not be loaded properly if the file is moved or renamed, or if the file is not found.

**Table B.1: File placement for local execution**

| File or folder | Location | Purpose |
|---|---|---|
| index.html | Project root | Contains the main dashboard, activity interface markup and script references. |
| styles.css | Project root | Contains the shared interface styling and responsive layout rules. |
| app.js | Project root | Manages navigation, shared state, theme controls, help content and sound functions. |
| server.js | Project root | Starts the local HTTP server and delivers the application files. |
| three-engine.js | Project root | Contains the integrated Three.js dashboard-background engine. |
| package.json | Project root | Defines project information and the available Node.js commands. |
| package-lock.json | Project root | Records the resolved versions of the Node.js dependencies. |
| games | Project root | Contains the five JavaScript activity files loaded by the platform. |
| bubble-sort.js | games folder | Contains the Bubble Sort activity logic. |
| dijkstra.js | games folder | Contains the Dijkstra's Pathfinding activity logic. |
| tic-tac-toe.js | games folder | Contains the Tic-Tac-Toe activity logic. |
| three-mens-morris.js | games folder | Contains the Three Men's Morris activity logic. |
| chess.js | games folder | Contains the Chess activity logic. |

## B.2 Starting the Local Server

This computer that is used to run the platform locally must have node installed. The project files provided do not contain a required node version. After the user checks the correct organization of the files, he/she needs to change to the project root folder and type the following command in the terminal:

node server.js

This terminal window should remain open while the platform is being used. The server is configured to use port 3000, so the platform can then be opened in a browser at:

http://localhost:3000

The following commands can also be used because the scripts in package.json both execute node server.js:

npm start

or:

npm run dev

Express is listed as a dependency in package.json, but no sign that the supplied server.js actually relies on Express; rather it makes use of the built-in http, fs and path modules of node. Therefore, it is possible to start the current server without a need to install Express. npm install can be used to recreate the local node_modules dependencies for the supplied local server if the directory is missing, but should not be required otherwise as this is specified in package-lock.json.

After the platform is opened, the user will be able to select an activity from the dashboard. When the activity fails to load, the user should check the following: If it exists, a JavaScript file should be located in a games folder and have the same name as the script path in the index.html file.

The local server can be stopped by returning to the terminal and pressing:

Ctrl+C

### B.3 Local Storage and Troubleshooting

This platform stores selected information in the browser's localStorage. The following keys are used:

- comp_thinking_state stores the shared score, stored level values and selected game statistics.
- app_theme stores the selected light or dark theme.
- ct_help_seen records whether the automatic help instructions have already been displayed for each activity.

Information stored should be accessed using the same browser and the same local address (including port number). By deleting site data, you will delete these records from the browser.

This unfinished session values such as active game boards, Dijkstra timers, move histories, Chess undo data etc. are not stored. By reloading the page, a new game session is started instead, rather than the previous activity state.

This platform should be accessed via http://localhost:3000 and not by directly opening the file index.html. The local server sets up the typical browser environment for the application files and scripts to load.

If port 3000 is used, the other process that has it for communication should be terminated or the PORT value in server.js should be set to a port that is not in use. The browser address needs to be the same port number as the changed port number.

There are some external resources that you need to be connected to the internet to access. This makes Google Fonts being loaded from an external service, and the intended dependency for the Three.js library being loaded from a content delivery service via js Delivr. The continuity of use has not been demonstrated and it is not possible to use offline.

If background (Three.js) or other network resource (loaded from somewhere else) is not visible, make sure the platform is opened via the local Node.js server and that the appropriate network resource is available. In the environment tested, the recorded functional test showed a normal background initialization.

# APPENDIX C: SYSTEM SCREENSHOTS

Appendix C presents screenshots of the platform dashboard, activity interfaces, instruction windows and selected functional-test results. The instruction screenshots show the guidance aprovided to learners before or during each activity. These screenshots support the implementation and testing discussions presented in Chapter Four.

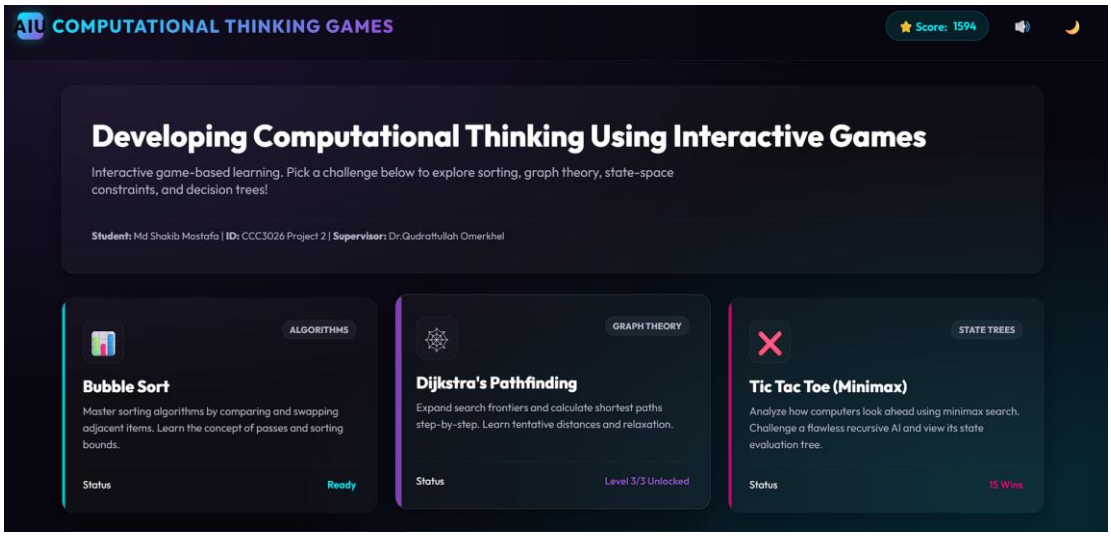



**Figure C.1:** Multi-Game Learning Platform Dashboard

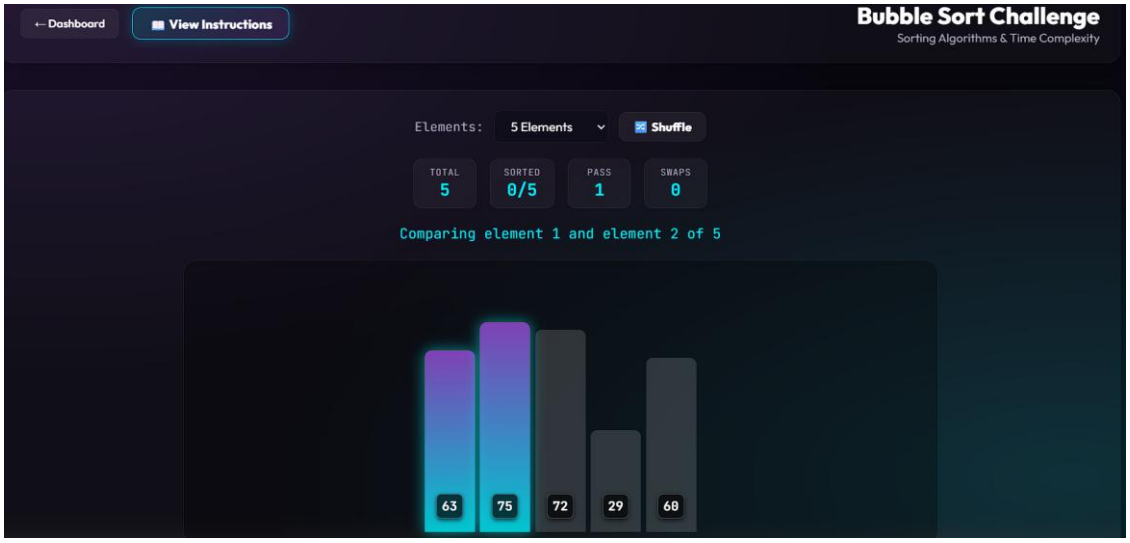



**Figure C.2:** Bubble Sort Activity

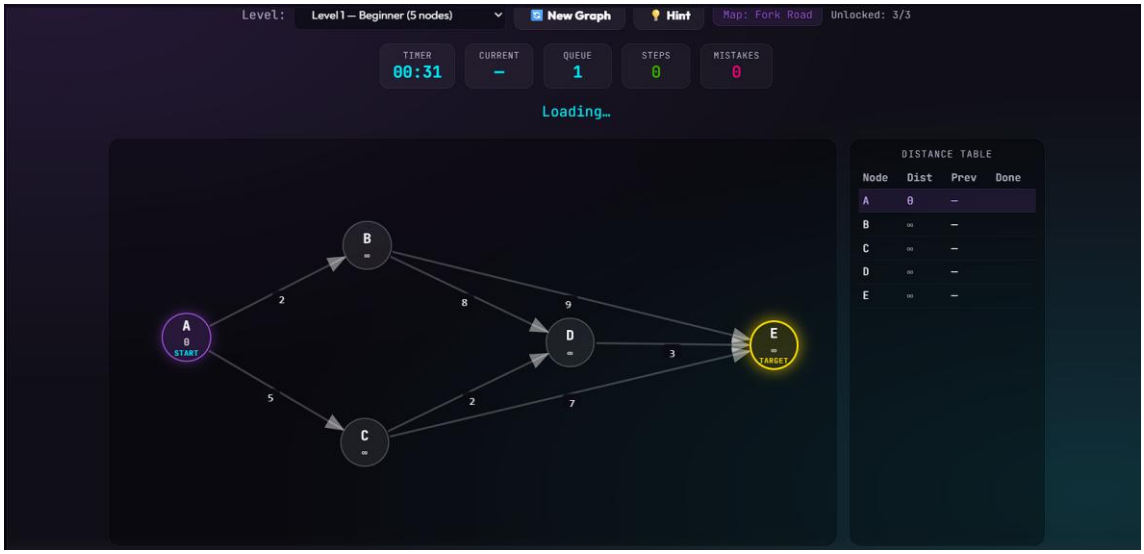



**Figure C.3:** Dijkstra's Pathfinding Activity

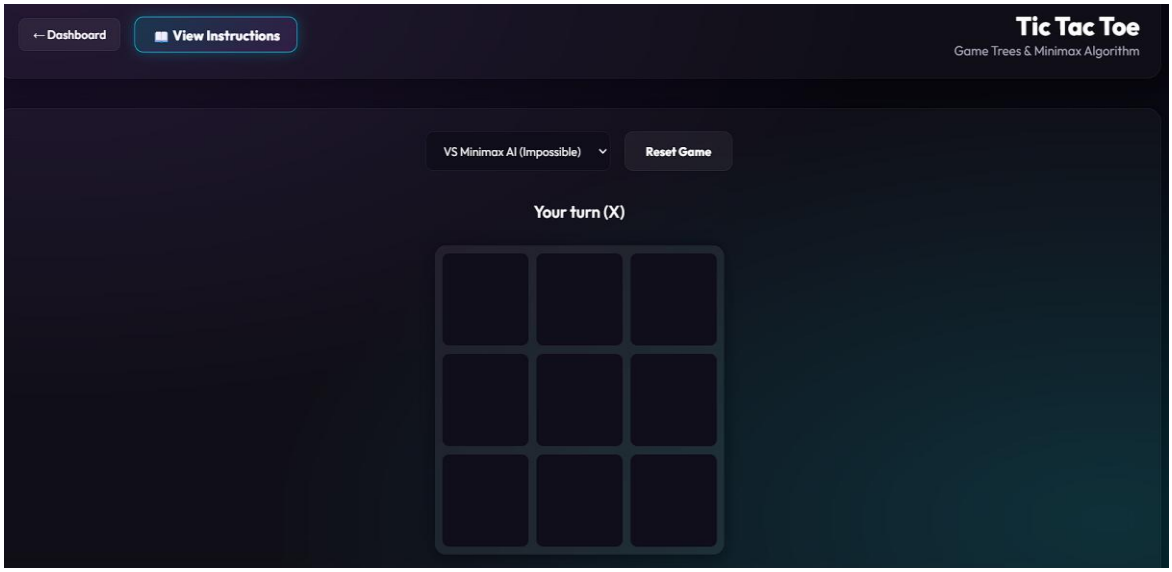



**Figure C.4:** Tic-Tac-Toe Minimax Evaluation Panel

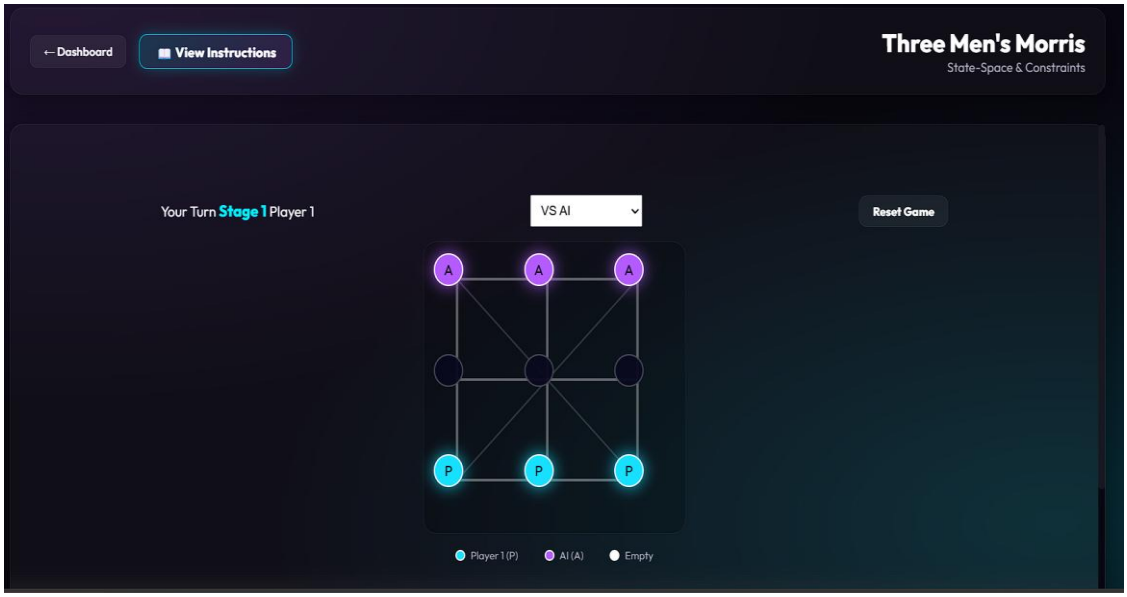



**Figure C.5:** Three Men's Morris Activity

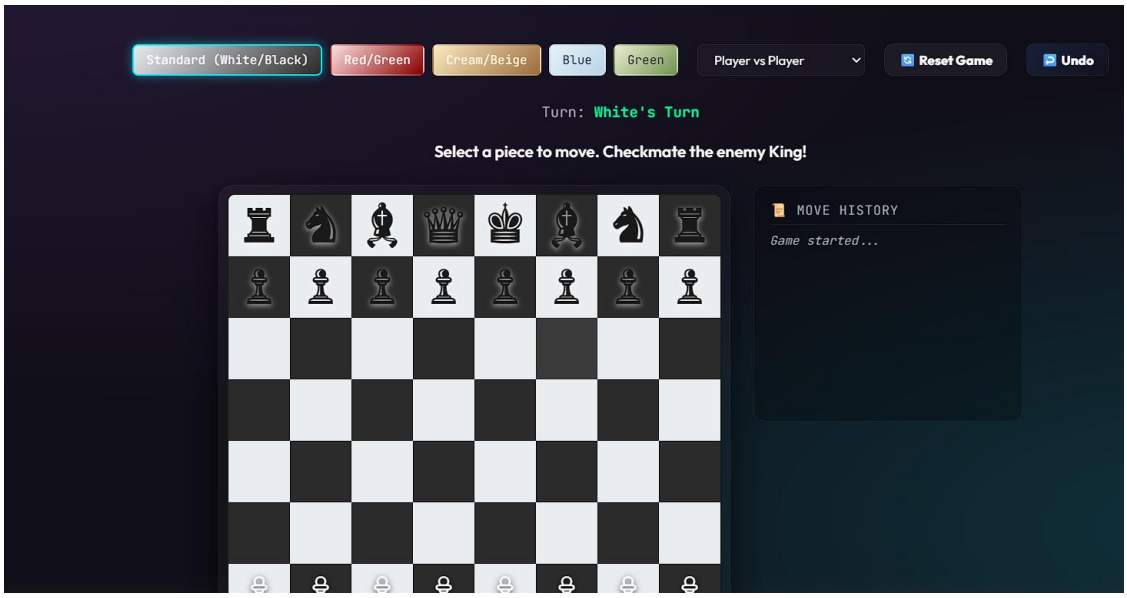



**Figure C.6:** Chess Activity

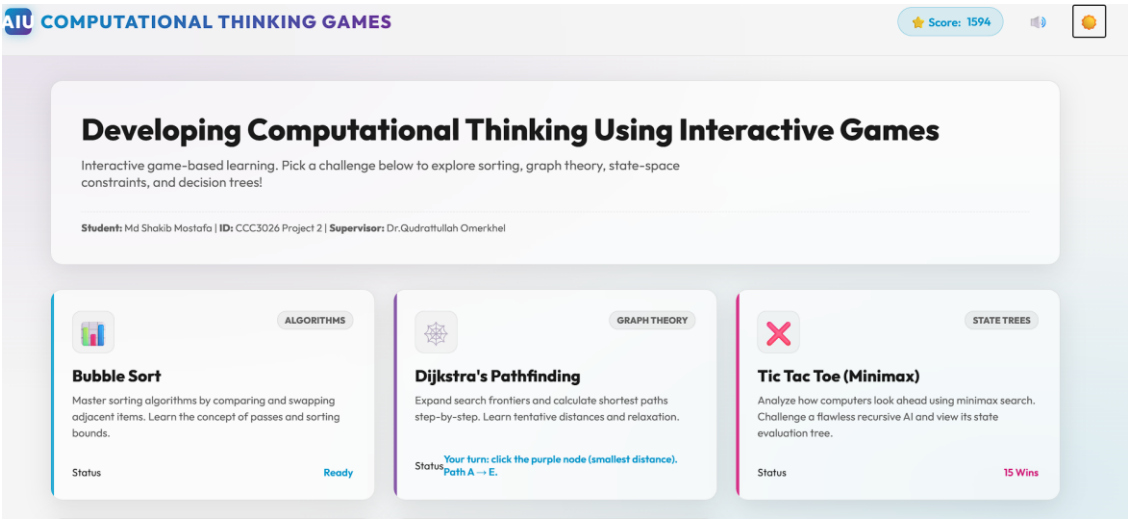



**Figure C.7:** Light and Dark Theme Interfaces